

Lecture 6

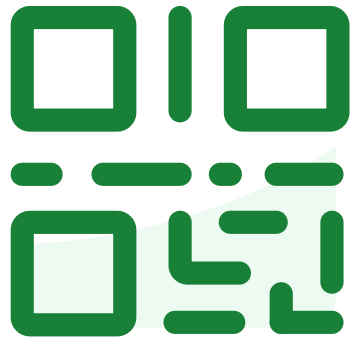
GMM Recap + Linear Regression (1)

Getting started with linear regression

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#3689302**

① The Slido app must be installed on every computer you're presenting from

slido

Roadmap

- Gaussian Mixture Model
- Linear Regression Formulation
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation



3689302

Gaussian Mixture Model

- Gaussian Mixture Model
- Linear Regression Formulation
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation



3689302

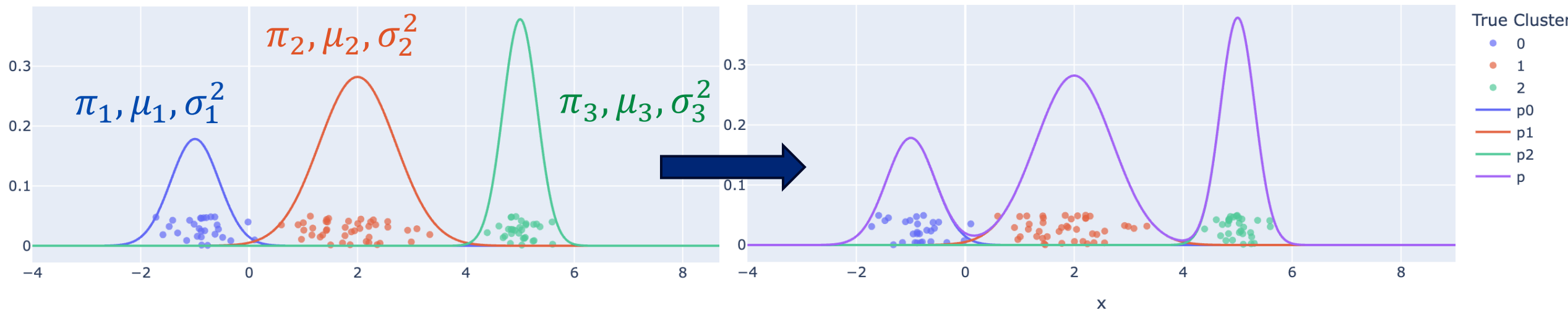
Gaussian Mixture Model (GMM)



3689302

The Gaussian mixture model defines the probability of sampling a D -dimensional vector $\mathbf{x}$ as a **weighted combination** of Gaussians:

- **Example:** $K=3$





3689302

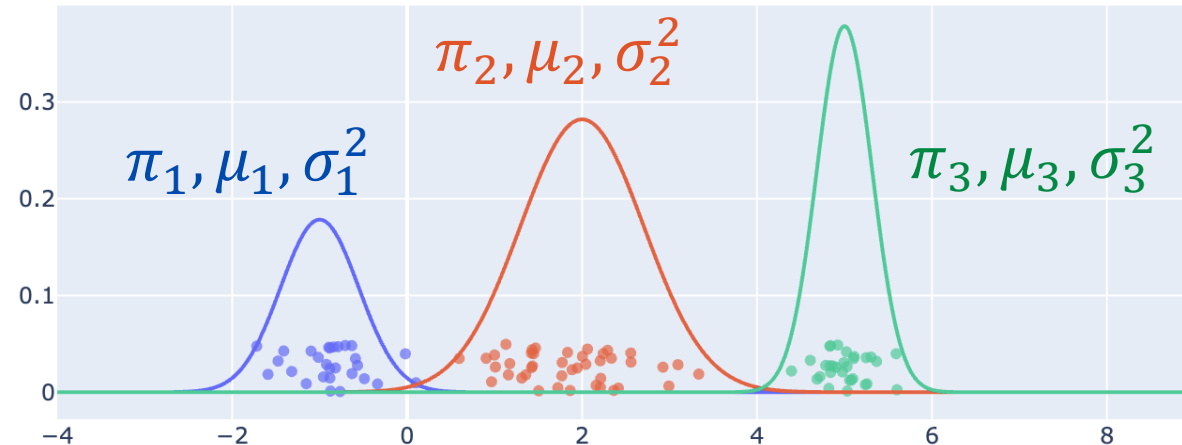
Gaussian Mixture Model (GMM)

The Gaussian mixture model defines the probability of sampling a D -dimensional vector $\mathbf{x}$ as a **weighted combination** of Gaussians:

$$p(\mathbf{x} \mid \pi, \mu_1, \Sigma_1, \dots, \mu_K, \Sigma_K) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \mu_k, \Sigma_k)$$

where $\sum_{k=1}^K \pi_k = 1$ is a probability distribution over clusters.

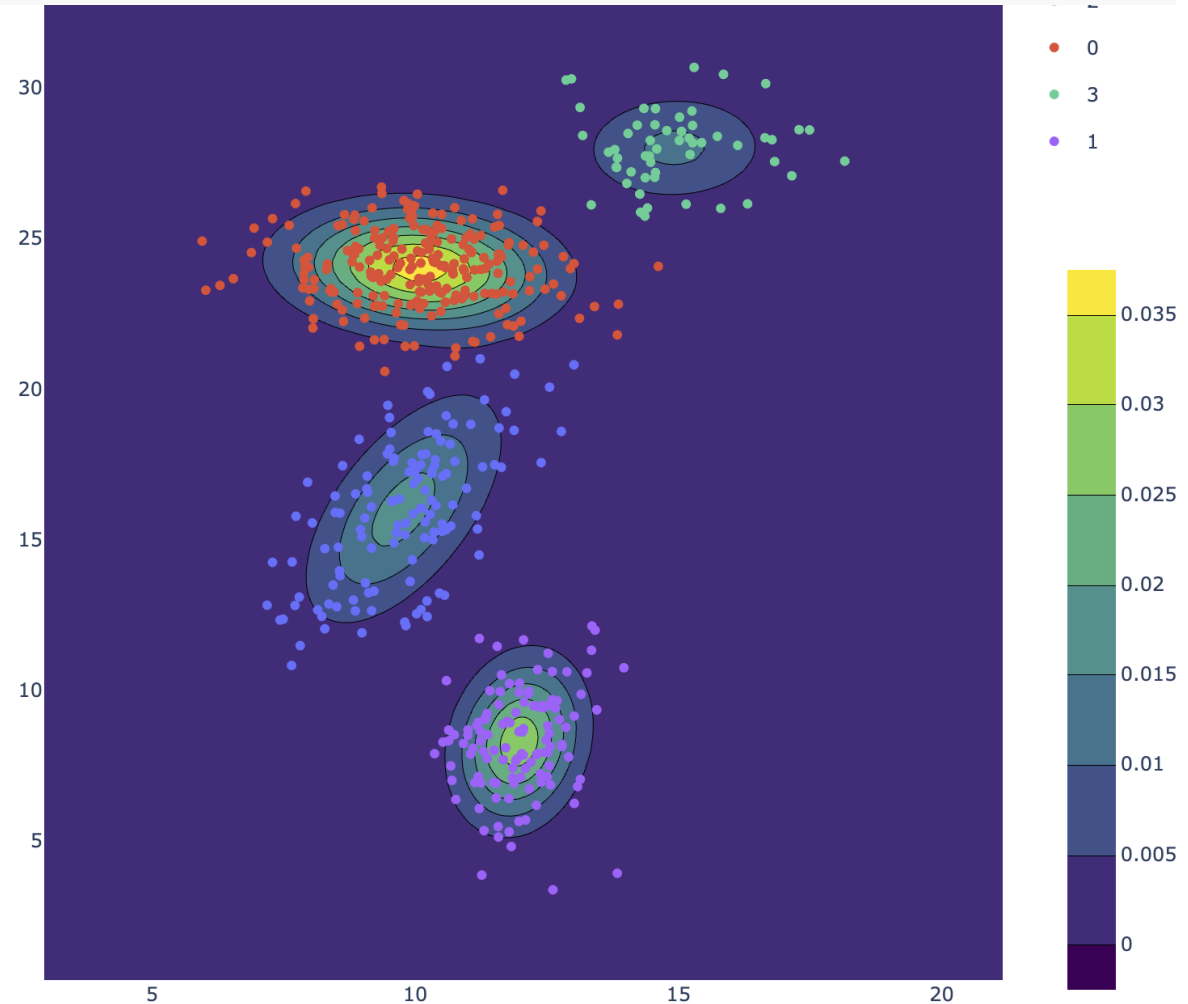
- **Example: $K=3$**



Demo

Gaussian Mixture Model

```
from sklearn.mixture import GaussianMixture
# Create a Gaussian Mixture Model with 4 components
gmm = GaussianMixture(n_components=4, random_state=42, )
# Fit the model to the data
gmm.fit(bikes[['Speed', 'Length']])
# Get the cluster labels
bikes['scikit gmm'] = gmm.predict(bikes[['Speed', 'Length']]).astype(str)
```



3689302



3689302

The GMM is a **Latent Variable Model**

We introduce a **latent** (unobserved) **assignment variable** z for each $\mathbf{x}$.

$$p(z) = \pi_z$$

We then define the likelihood of $\mathbf{x}$ given z as:

$$p(\mathbf{x} | z) = \mathcal{N}(\mathbf{x} | \mu_z, \Sigma_z)$$

The joint probability of the $\mathbf{x}$ and z is then:

$$p(\mathbf{x}, z) = p(\mathbf{x} | z)p(z) = \mathcal{N}(\mathbf{x} | \mu_z, \Sigma_z)\pi_z$$

If we **marginalize over z** we obtain the **GMM**:

$$p(\mathbf{x}) = \sum_{k=1}^K p(\mathbf{x}, z = k) = \sum_{k=1}^K \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)\pi_k$$



3689302

The GMM is a Generative Model

A **generative model** is a model that can **generate the data**.

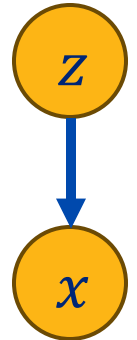
- The GMM is a generative model.

Recall, the joint distribution has the following factorization:

$$p(\mathbf{x}, z) = p(z) p(\mathbf{x} | z)$$

We can draw a **graph** where the parents of a variable are the variables in its **conditional distribution**.

- This is a **graphical model** and is used to represent the **conditional structure** of a probability model.



We can sample data from a graphical model using **ancestor sampling**.

- Sample all the variables that have no parents: $p(z)$
- Continue sampling variables after all their parents have been sampled: $p(\mathbf{x} | z)$



3689302

Demo

Sampling from a GMM

```
# Ancestor Sampling to create a synthetic dataset
np.random.seed(42)
N = 100

mu = np.array([-1, 2, 5])
pi = np.array([0.2, 0.5, 0.3])
Sigma = np.array([0.2, 0.5, .1])
```

✓ 0.0s

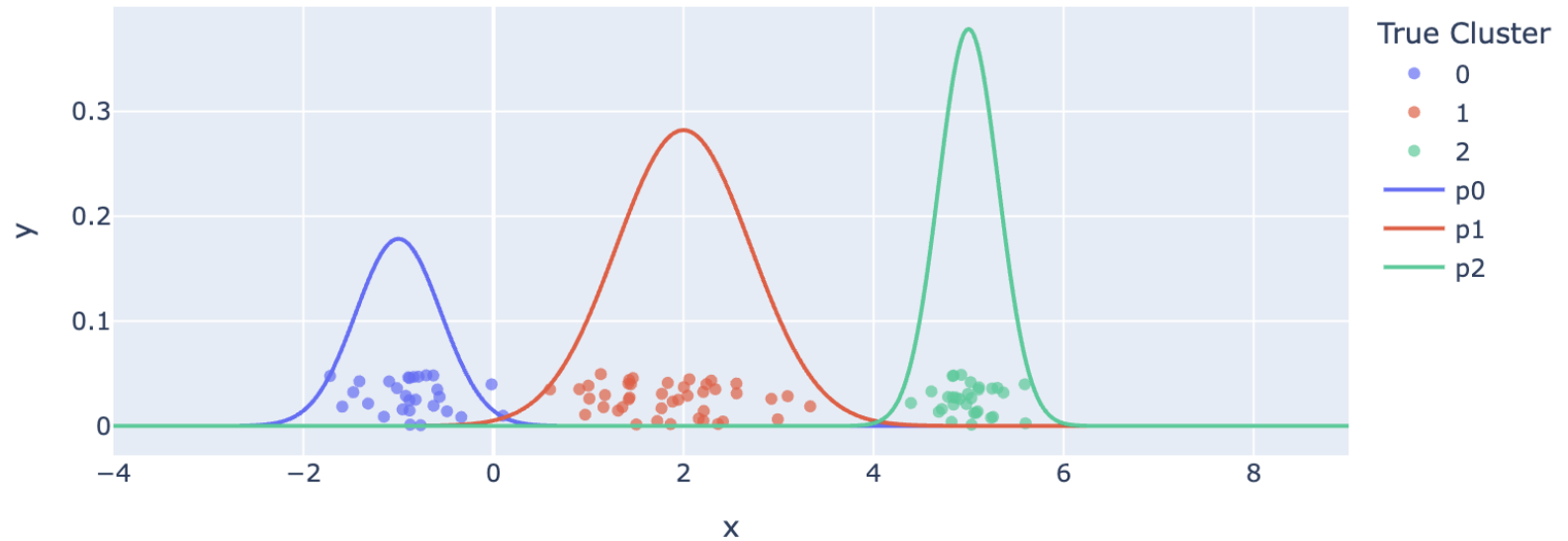
```
z = np.random.choice(len(mu), size=N, p=pi)
x = np.random.normal(mu[z], np.sqrt(Sigma[z]))
```

✓ 0.0s

```
log_likeli
p
a
```

✓ 0.0s

Synthetic Dataset from GMM (Log Likelihood: 181.82)



Latent Variable Posteriors

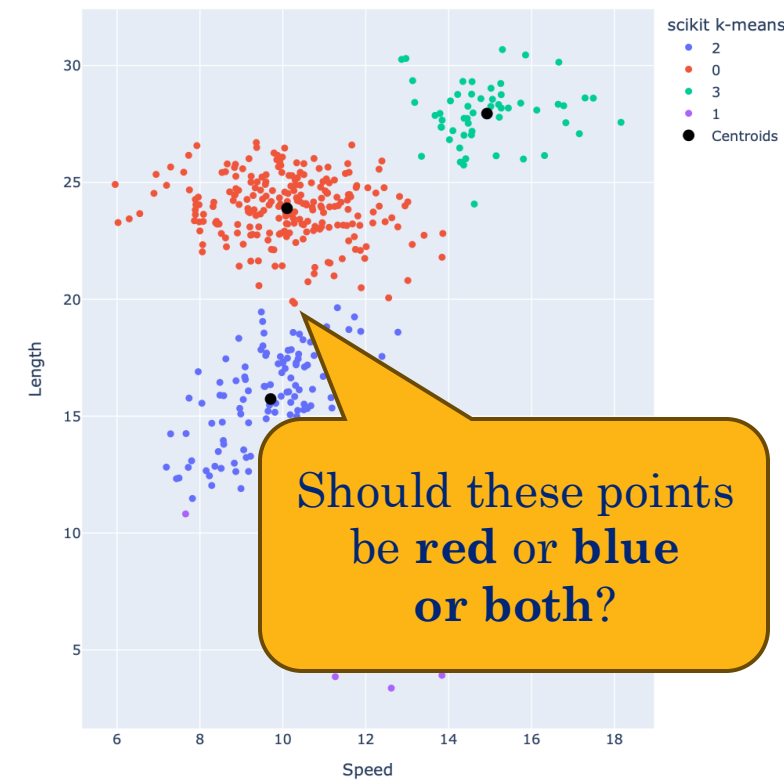
We are interested in modeling the **distribution** (*the uncertainty*) over the **cluster assignment** z .

- **Prior:** $p(z) = \pi_z$
- **Likelihood:** $p(\mathbf{x} | z) = \mathcal{N}(\mathbf{x} | \mu_z, \Sigma_z)$

We can use Bayes rule to compute the posterior:

$$p(z | \mathbf{x}, \pi, \{\mu_k, \Sigma_k\}_{k=1}^K) = \frac{p(\mathbf{x} | z)p(z)}{\sum_{k=1}^K p(\mathbf{x} | z = k)p(z = k)} = \frac{\mathcal{N}(\mathbf{x} | \mu_z, \Sigma_z)\pi_z}{\sum_{k=1}^K \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k)\pi_k}$$

If we **could estimate** $\pi, \{\mu_k, \Sigma_k\}_{k=1}^K$ **from the data** we could compute this distribution over cluster assignments.





3689302

Estimating the GMM Parameters

We could try to use the method of **maximum likelihood**:

$$\ln \left(p(\{\mathbf{x}_n\}_{n=1}^N \mid \pi, \{\mu_k, \Sigma_k\}_{k=1}^K) \right) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n \mid \mu_k, \Sigma_k) \right)$$

- however, the summation inside of the log couples the parameters.

But, if we extend the model to include the **latent variable** z

$$\ln \left(p(\{\mathbf{x}_n, z_n\}_{n=1}^N \mid \pi, \{\mu_k, \Sigma_k\}_{k=1}^K) \right) = \sum_{n=1}^N \left(\ln(\pi_{z_n}) + \ln \left(\mathcal{N}(\mathbf{x}_n \mid \mu_{z_n}, \Sigma_{z_n}) \right) \right)$$

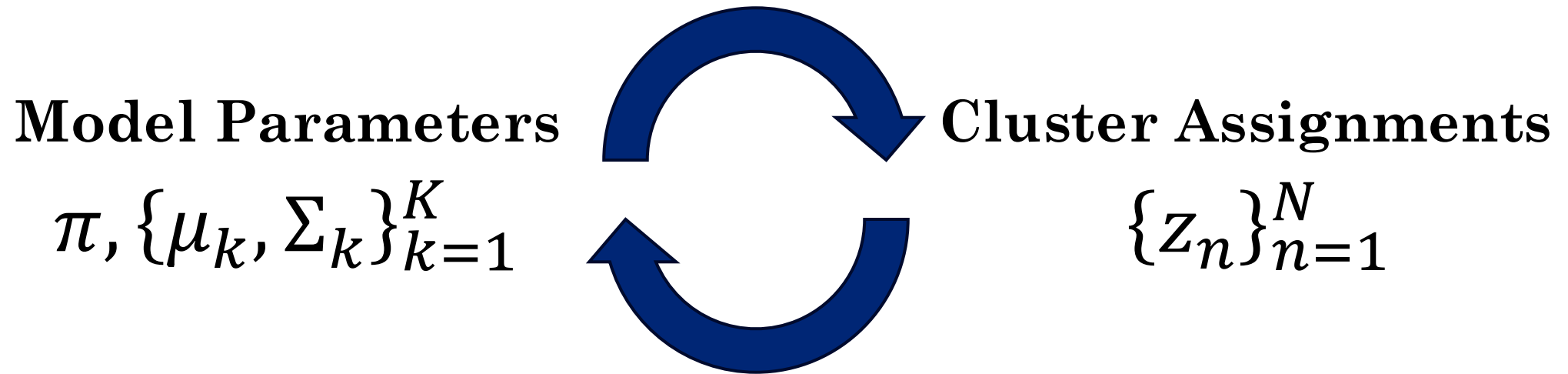
- We can optimize the parameters directly (sum of log terms).
- But we don't know the values of $\{z_n\}_{n=1}^N$

Quick Recap



3689302

If we knew the model parameters, we could easily compute the cluster assignments.



If we knew the cluster assignments, we could easily estimate the model parameters.

How can we solve this cyclic dependency?



3689302

The EM Algorithm

The **Expectation-Maximization (EM)** algorithm is a standard technique for fitting **latent variable** models like the Gaussian Mixture Model.

1. **Initialize** the model parameters θ .
2. **Iterate** until convergence.
 - **E-Step:** Compute expected log-likelihood.

$$Q(\theta') = \sum_Z \ln(p(X, Z | \theta')) p(Z | X, \theta)$$

Easy to optimize
joint probability

Current distribution
over the latent Z

- **M-Step:** Maximize the expected log-likelihood.

$$\theta = \arg \max_{\theta'} Q(\theta')$$

Updates distribution over Z .



3689302

The EM Algorithm: **E**-step

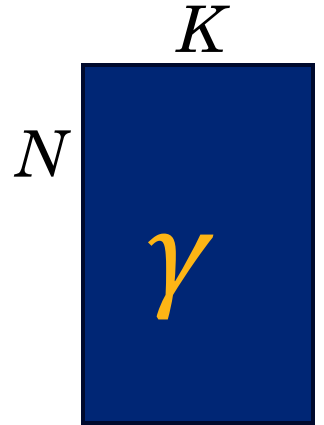
E-Step: We don't know the value of $\{z_n\}_{n=1}^N$ but we can compute

$$\gamma_{nk} = p(z_n = k | \mathbf{x}_n, \pi, \{\mu_k, \Sigma_k\}_{k=1}^K) = \frac{p(\mathbf{x}_n | z_n) p(z_n)}{\sum_{k'=1}^K p(\mathbf{x}_n | k') p(k')} = \frac{\mathcal{N}(\mathbf{x}_n | \mu_{z_n}, \Sigma_{z_n}) \pi_{z_n}}{\sum_{k'=1}^K \mathcal{N}(\mathbf{x}_n | \mu_{k'}, \Sigma_{k'}) \pi_{k'}}$$

- γ_{nk} are “soft” assignments to each class.

Using the γ_{nk} we can compute the **expected log likelihood**:

$$\begin{aligned} Q(\pi, \{\mu_k, \Sigma_k\}_{k=1}^K) &= \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} \ln \left(p(\mathbf{x}_n, z_n = k | \pi, \{\mu_k, \Sigma_k\}_{k=1}^K) \right) \\ &= \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\ln(\pi_k) + \ln(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))) \end{aligned}$$





3689302

The EM Algorithm: M-Step (π_k)

M-Step: Compute the parameters that maximize Q :

$$Q(\pi, \{\mu_k, \Sigma_k\}_{k=1}^K) = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\ln(\pi_k) + \ln(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)))$$

Optimizing with respect to π_k :

Lagrangian for the
normalization constraint

$$\frac{\partial}{\partial \pi_k} \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\ln(\pi_k) + \ln(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))) + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) = \sum_{n=1}^N \frac{\gamma_{nk}}{\pi_k} + \lambda = 0$$

$$\begin{aligned} 1 &= \sum_{k=1}^K \pi_k = -\frac{1}{\lambda} \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} = -\frac{1}{\lambda} \sum_{n=1}^N 1 \\ &= -\frac{N}{\lambda} \Rightarrow \lambda = -N \end{aligned}$$

$$\begin{aligned} \pi_k &= -\frac{1}{\lambda} \sum_{n=1}^N \gamma_{nk} \\ &= \frac{1}{N} \sum_{n=1}^N \gamma_{nk} = \frac{N_k}{N} \end{aligned}$$

$$N_k = \sum_{n=1}^N \gamma_{nk}$$



3689302

The EM Algorithm: **M-Step** (μ_k)

M-Step: Compute the parameters that maximize Q :

$$Q(\pi, \{\mu_k, \Sigma_k\}_{k=1}^K) = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\ln(\pi_k) + \ln(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)))$$

Optimizing with respect to μ_k :

$$\frac{\partial}{\partial \mu_k} \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\ln(\pi_k) + \ln(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))) = \sum_{n=1}^N \gamma_{nk} \Sigma_k^{-1} (\mathbf{x}_n - \mu_k) = 0$$

$$\mu_k = \frac{1}{\sum_{n=1}^N \gamma_{nk}} \sum_{n=1}^N \gamma_{nk} \mathbf{x}_n = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \mathbf{x}_n$$



3689302

The EM Algorithm: **M-Step** (Σ_k)

M-Step: Compute the parameters that maximize Q :

$$Q(\pi, \{\mu_k, \Sigma_k\}_{k=1}^K) = \sum_{n=1}^N \sum_{k=1}^K \gamma_{nk} (\ln(\pi_k) + \ln(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)))$$

Optimizing with respect to Σ_k requires matrix calculus (out of scope):

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T$$

Summarizing the other parameters:

$$N_k = \sum_{n=1}^N \gamma_{nk}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \mathbf{x}_n$$



3689302

The EM Algorithm for GMMs

1. **Initialize** the model parameters $\pi, \{\mu_k, \Sigma_k\}_{k=1}^K$ (often using k-means)
2. **Iterate** until convergence
 - **E-Step:** compute expected log-likelihood

$$\gamma_{nk} = \frac{\mathcal{N}(\mathbf{x}_n | \mu_{z_n}, \Sigma_{z_n}) \pi_{z_n}}{\sum_{k'=1}^K \mathcal{N}(\mathbf{x}_n | \mu_{k'}, \Sigma_{k'}) \pi_{k'}}$$

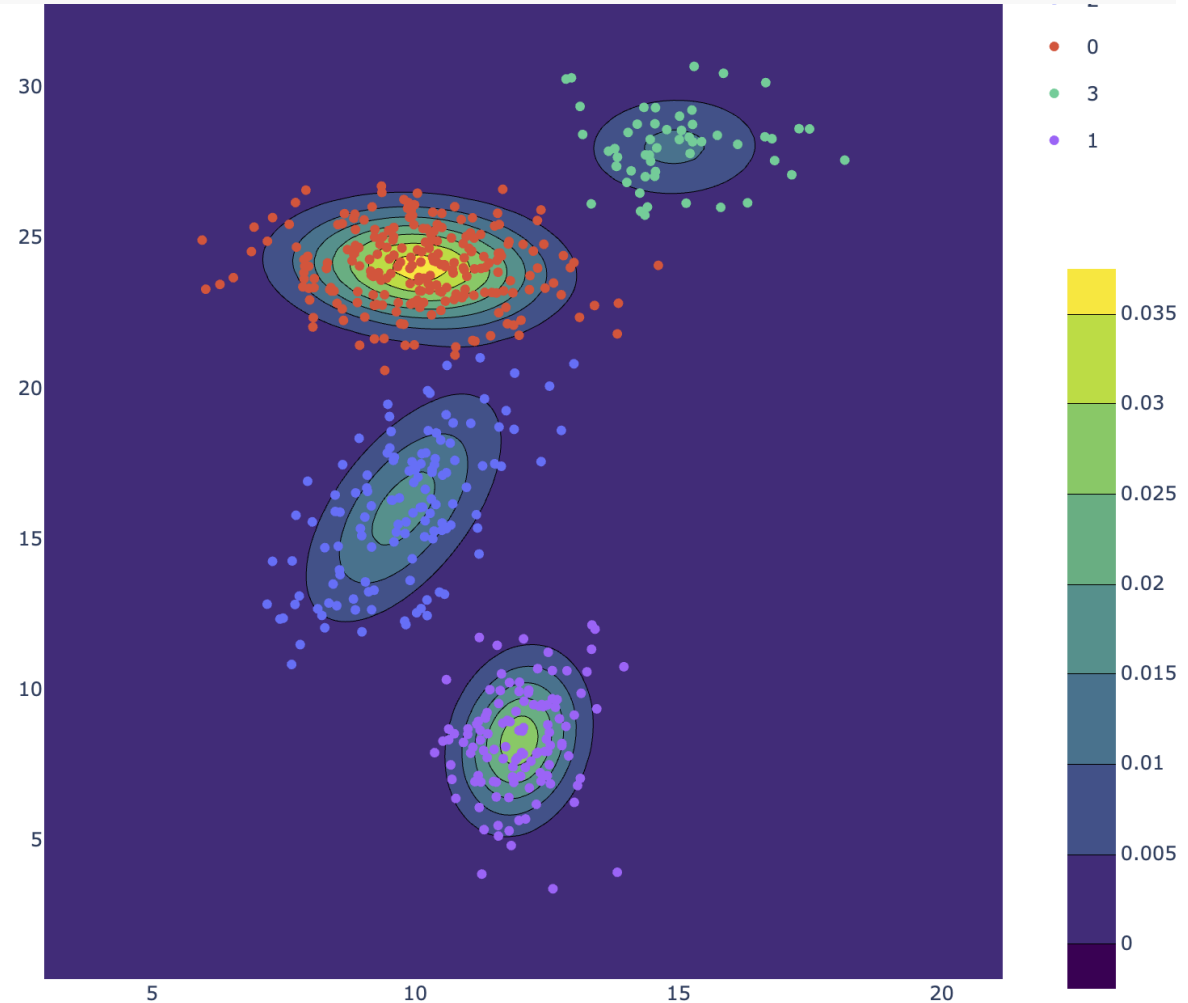
- **M-Step:** maximize the expected log-likelihood

$$N_k = \sum_{n=1}^N \gamma_{nk}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} \mathbf{x}_n$$
$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T$$

Demo

Implementing EM for GMMs

```
from sklearn.mixture import GaussianMixture
# Create a Gaussian Mixture Model with 4 components
gmm = GaussianMixture(n_components=4, random_state=42, )
# Fit the model to the data
gmm.fit(bikes[['Speed', 'Length']])
# Get the cluster labels
bikes['scikit gmm'] = gmm.predict(bikes[['Speed', 'Length']]).astype(str)
```



3689302

Linear Regression

- Gaussian Mixture Model
- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

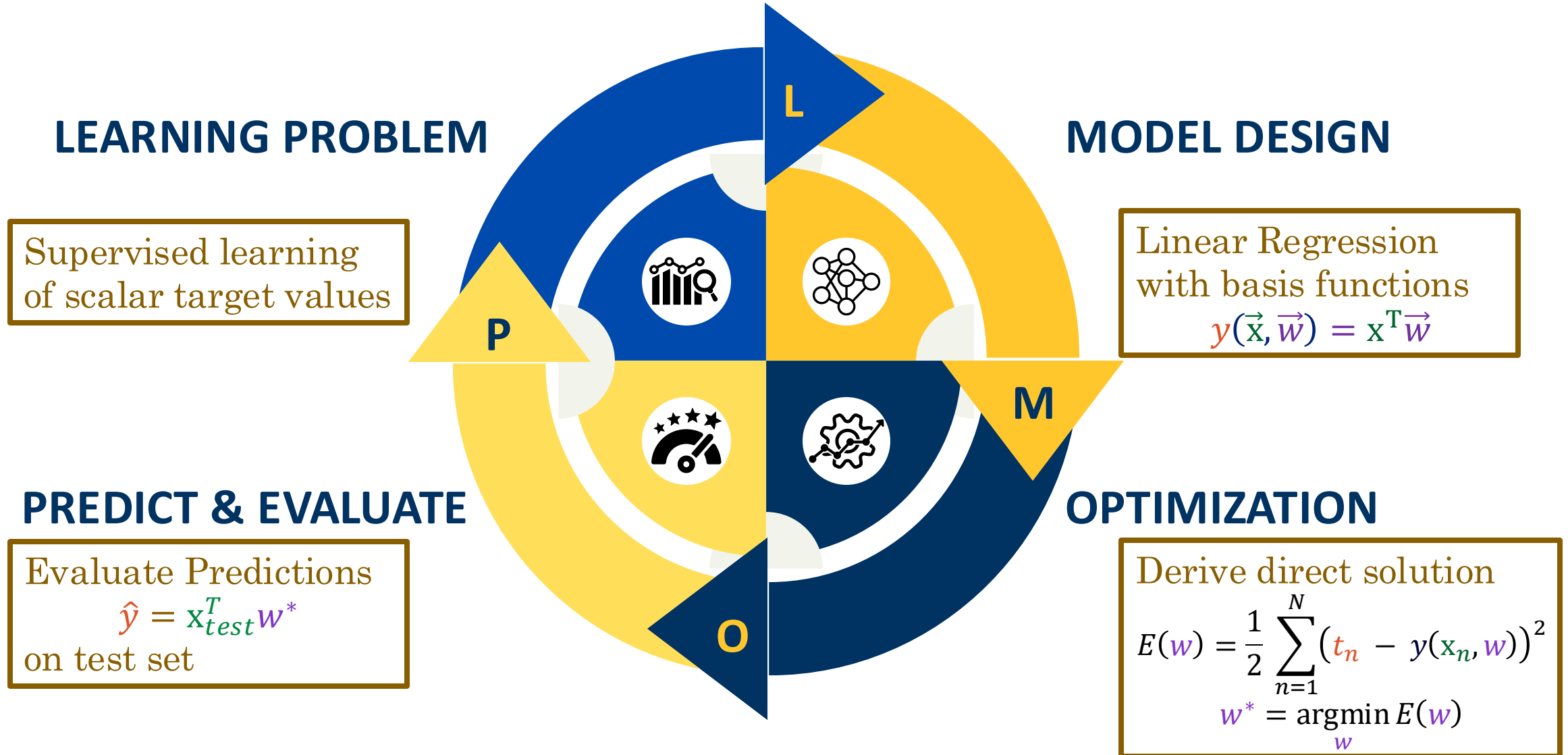


3689302

Linear Regression Outline



3689302



Learning Problem



3689302

LEARNING PROBLEM

Supervised learning
of scalar target values

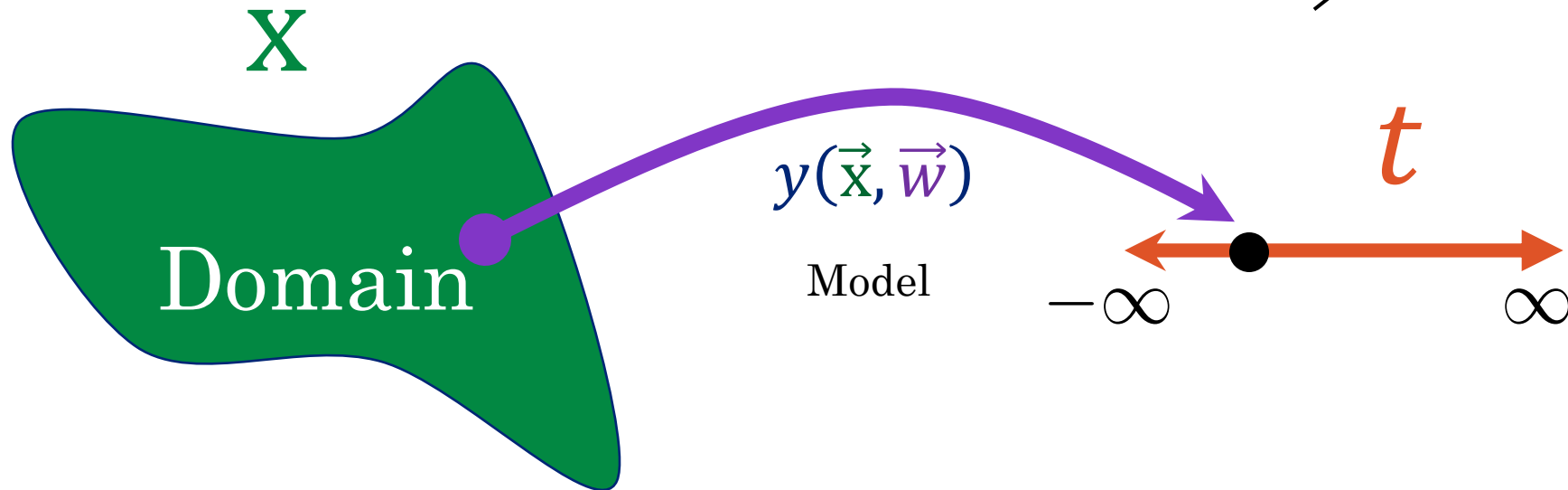
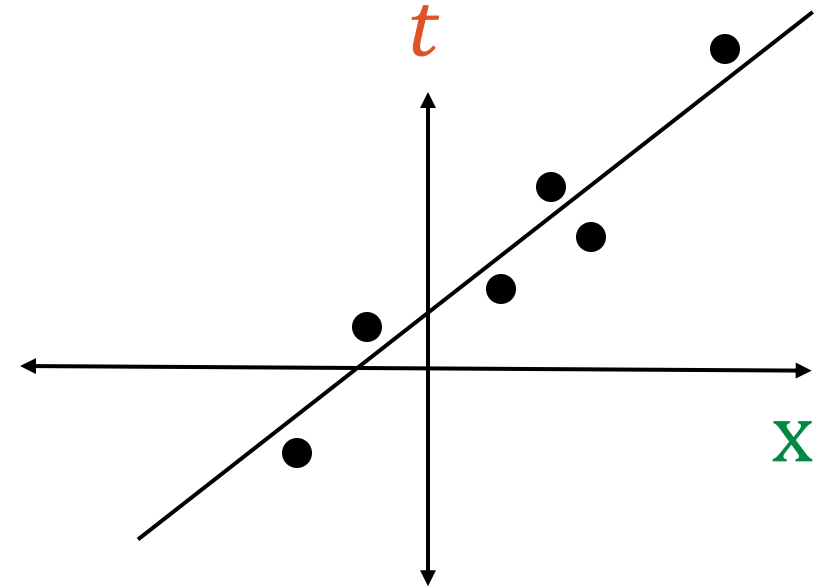


Regression



3689302

- Estimating relationship between $\mathbf{x}$ and t .
 - t is a quantitative value
 - We will soon see $\mathbf{x}$ can be almost anything ...



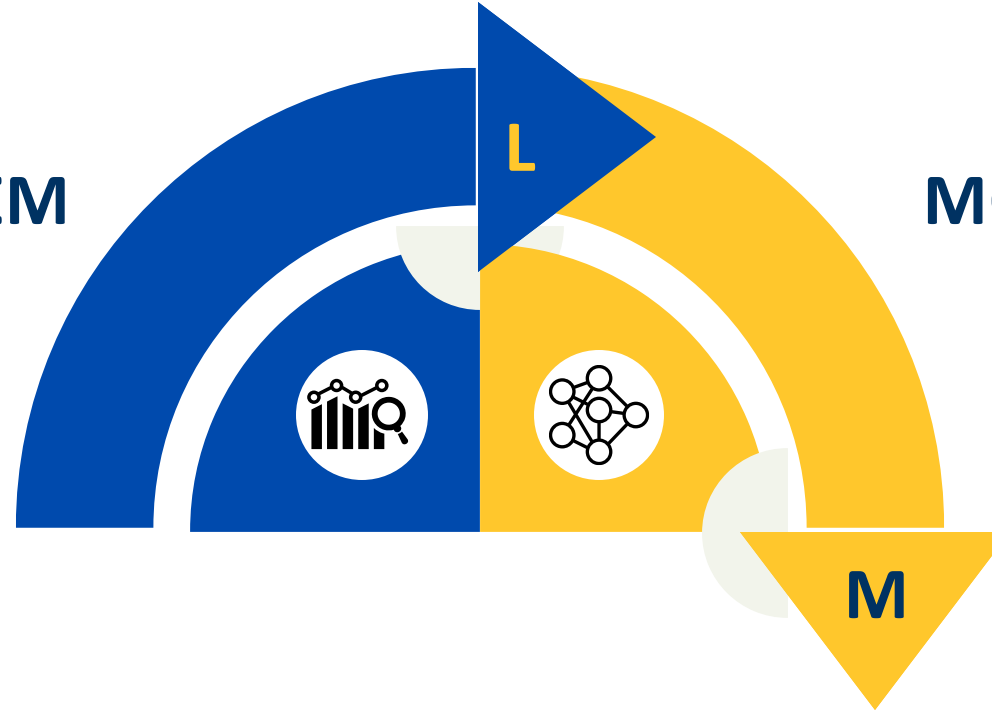
Model Design



3689302

LEARNING PROBLEM

Supervised learning
of scalar target values



MODEL DESIGN

Linear Regression
with basis functions
 $y(\vec{x}, \vec{w}) = \vec{x}^T \vec{w}$

Supervised Linear Regression



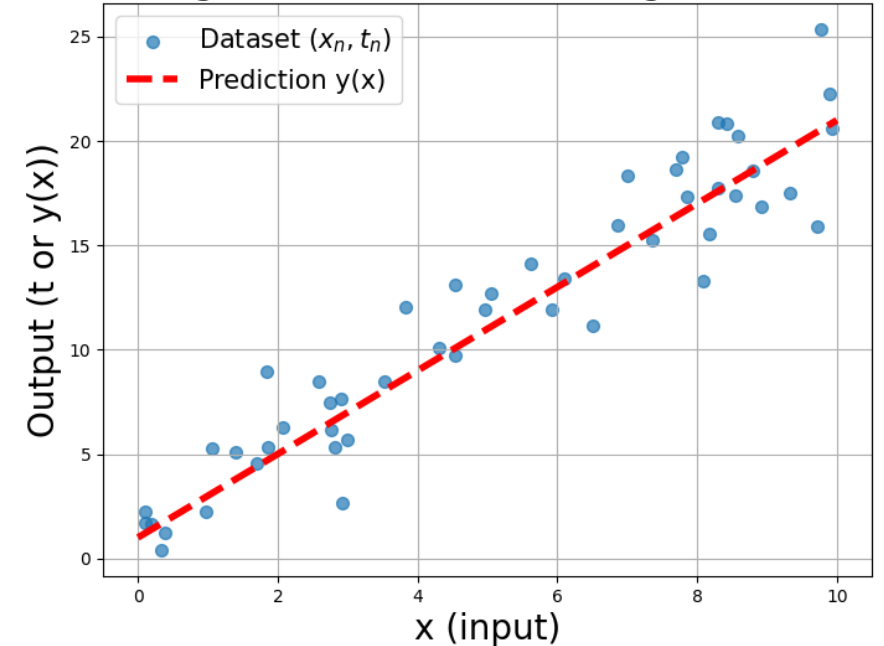
3689302

- Goal: Predict one or more continuous *target* variable t given the vector $\vec{x} \in \mathbb{R}^D$ of *input* variables.

$$\{(x_n)\}_{n=1}^N \xrightarrow{\vec{w}} \{(t_n)\}_{n=1}^N$$

- We formulate a function $y(\vec{x}, \vec{w})$ whose values for new input $\vec{x}$ constitute the predictions for the corresponding values of t .
- $\vec{w}$ is a vector of parameters that can be learned from the training data.

Visualizing Dataset and a Linear Regression Function



The Simplest Linear Regression Model



3689302

- The simplest model is of the following form:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D \quad \text{where } \vec{x} = (x_1, x_2, \dots, x_D)^T \in \mathbb{R}^D.$$

↓
Predicted output Linear combination of parameters
 $w_0, w_1, \dots, w_D$, each scaled by x_d 's.

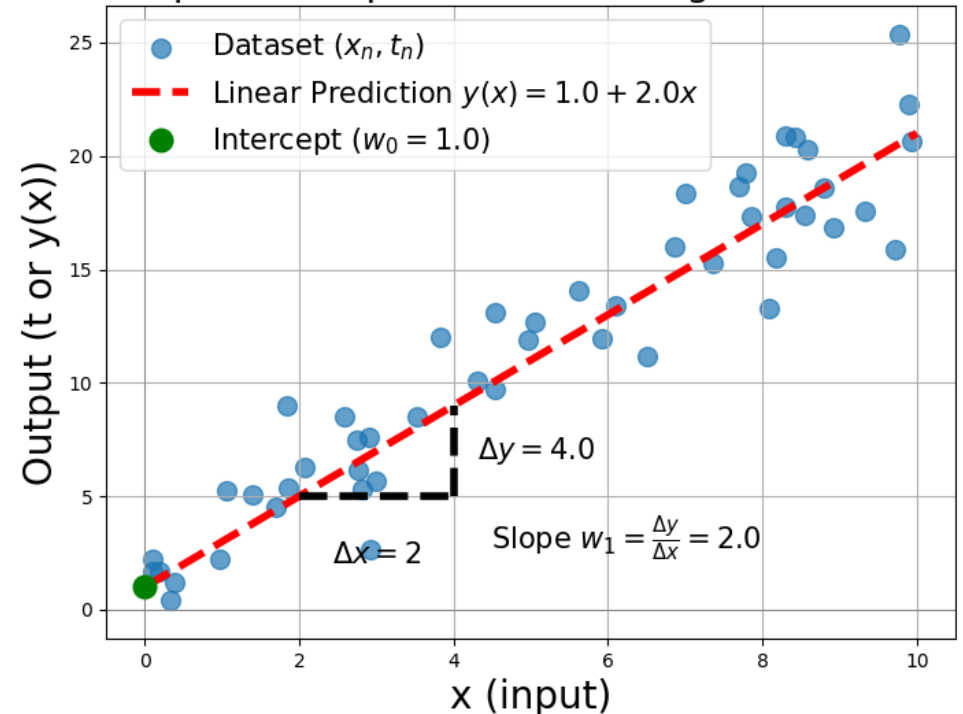
- In the simplest case of $D = 1$, we have:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1$$

↙
Intercept (shift)

↘
Slope (rate of change)

Intercept and Slope in a Linear Regression Function



The Simplest Linear Regression Model



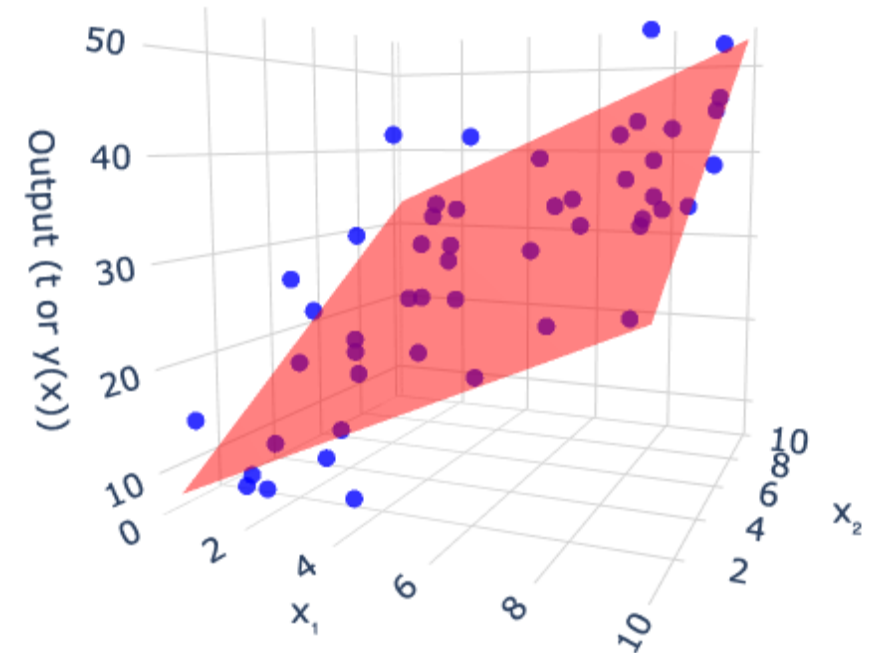
3689302

- The simplest model is of the following form:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D \quad \text{where } \vec{x} = (x_1, x_2, \dots, x_D)^T \in \mathbb{R}^D.$$

↓
Predicted output Linear combination of parameters
 $w_0, w_1, \dots, w_D$, each scaled by x_d 's.

- In higher dimension, the predicted values form a hyperplane in $\mathbb{R}^{D+1}$.



[Interactive link](#)



Which of the following is a linear regression model?

Basis Functions

- Gaussian Mixture Model
- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation



3689302

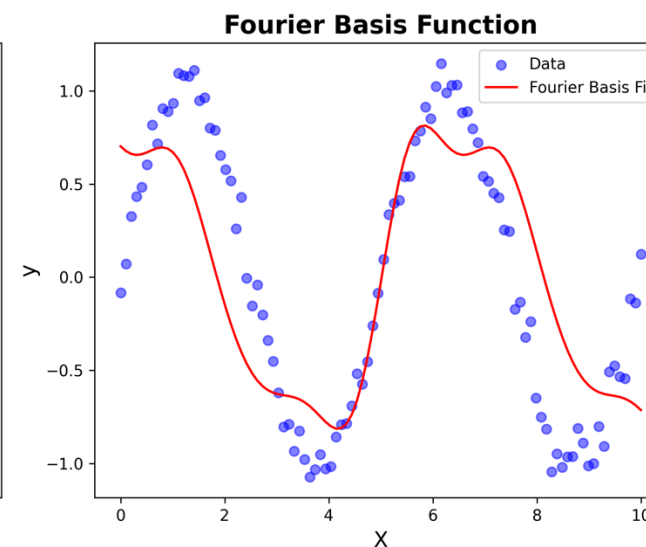
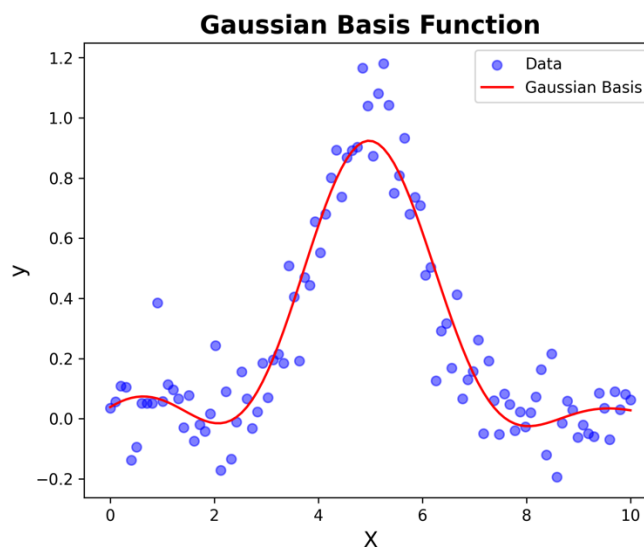
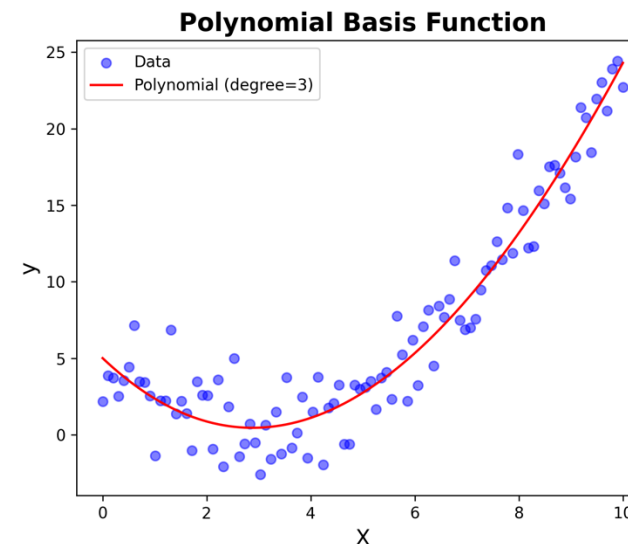
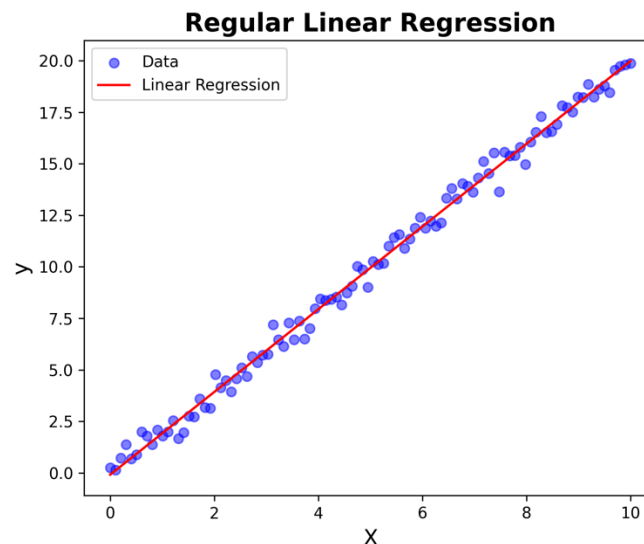
Linear Functions From Slido

These are all linear models with different basis functions.

We will now see what basis functions are...



3689302





3689302

What Does It Mean To Be a Linear Model?

In what sense are the previous plots linearly modeled?

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x})$$

Are linear models linear in the

1. Features?
2. Parameters?



Are linear models linear in the



3689302

What Does It Mean To Be a Linear Model?

In what sense are the previous plots linearly modeled?

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\vec{x})$$

Linear in the Parameters

Feature Functions

Are linear models linear in the

1. Features?
2. Parameters?



3689302

Basis Functions

- We can extend the class of models by considering linear combinations of fixed nonlinear functions of the input variables, of the form:

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x})$$

where $\phi_j(\mathbf{x})$ are known as basis functions.



3689302

More on Basis Function

- **Polynomial** basis: $\phi_j(x) = x^j$
- **Radial Basis Functions (RBF)** such as **Gaussian**: $\phi_j(x) = e^{-\frac{(x-\mu_j)^2}{2\sigma^2}}$
- **Sigmoidal** basis: $\phi_j(x) = \sigma\left(\frac{x-\mu_j}{\sigma}\right)$ where $\sigma(a) = \frac{1}{1+e^{-a}}$
 - $\tanh(a) = 2\sigma(2a) - 1$
- **Sinusoidal** basis: $\phi_j(x) = \sin x_j$ or $\phi_j(x) = \cos x_j$

Basis Functions as Features

Why did we care about basis functions before deep learning?

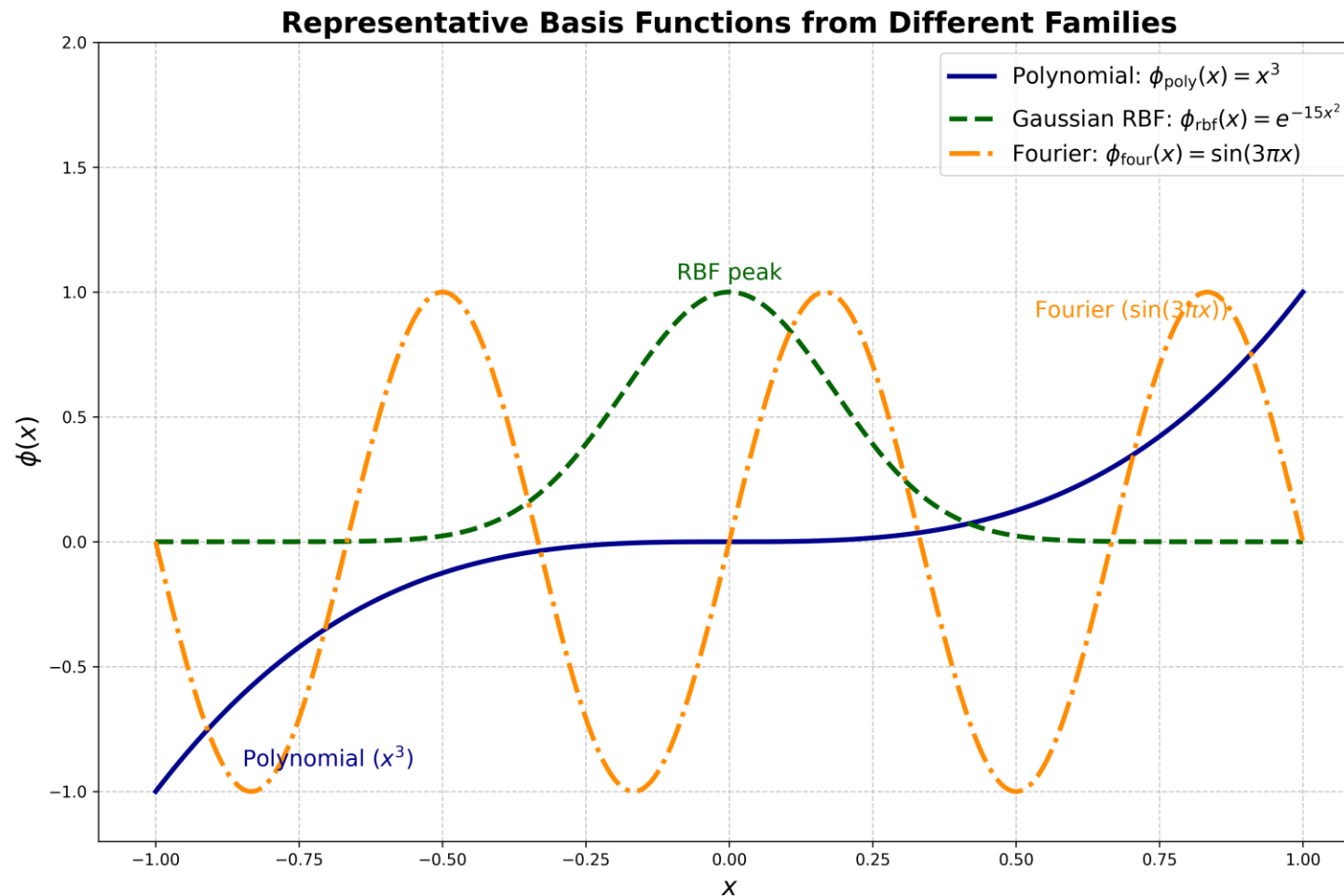
Fixed feature extraction was the norm before deep learning: raw input $x \rightarrow$ hand-designed features $\phi_j(x)$.

Goal: Pick expressive features so a simple linear model could succeed.

Popular choices: polynomials, radial-basis functions, Fourier terms, kernels, etc.
Heavy manual engineering.



3689302





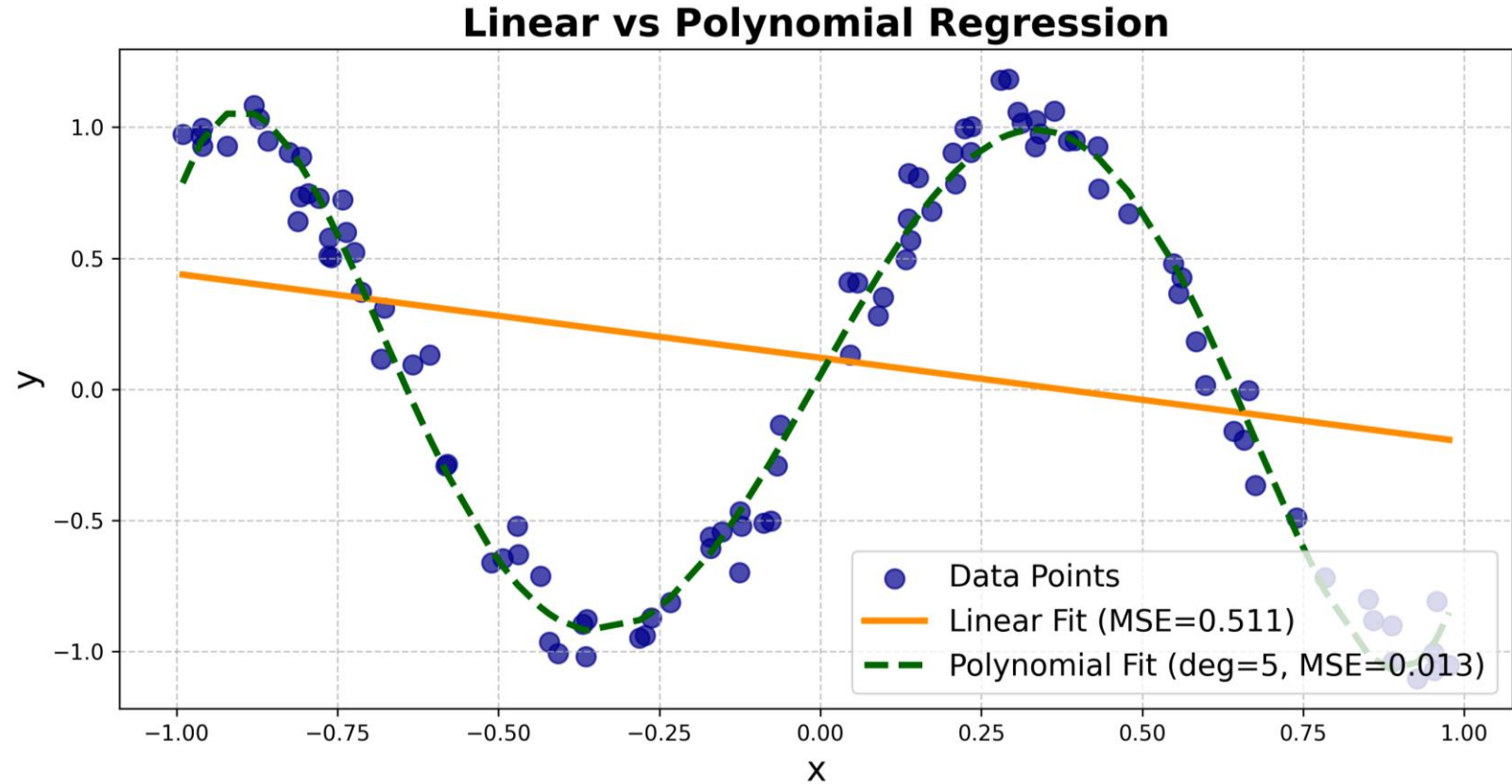
3689302

How Would Basis Functions Improve Predictions?

Using polynomial basis functions of degree 5, we redefined the linear regression equation as:

$$y(\vec{x}, \vec{w}) = \sum_{j=0}^5 w_j x^j$$

Looking at the Mean Squared Error (MSE) between targets and predictions, the polynomial fit has a better performance.



$$y(x) = 0.000 + 4.355x^1 - 0.203x^2 - 14.426x^3 + 0.164x^4 + 9.368x^5$$

$$y(x) = 0.120 - 0.320x$$

Vectorizing Calculations

- Gaussian Mixture Model
- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function
Minimization
- Geometric Interpretation
- Evaluation



3689302



3689302

Vectorizing Calculations

- Rewriting the equation for linear regression:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D = w_0 + \sum_{d=1}^D w_d x_d$$

- If we assume that there is an $x_0 = 1$, then the two terms above can be combined into one summation:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D = \sum_{d=0}^D w_d x_d$$

where $\vec{x} = (x_0, x_1, x_2, \dots, x_D)^T$ and $\vec{w} = (w_0, w_1, w_2, \dots, w_D)^T$

↓
Augmented input with x_0

↘
 $x_0 = 1$

Vectorizing Calculations



3689302

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + \dots + w_D x_D = \sum_{d=0}^D w_d x_d$$

where $\vec{x} = (x_0, x_1, x_2, \dots, x_D)^T$ and $\vec{w} = (w_0, w_1, w_2, \dots, w_D)^T$

$$\begin{aligned} y(\vec{x}, \vec{w}) &= \vec{w} \cdot \vec{x} \\ &= \vec{w}^T \vec{x} = \vec{x}^T \vec{w} \end{aligned}$$



3689302

Matrix Notation

- Assuming we have $n = 1, \dots, N$ samples in our dataset, we can write:


$$\begin{cases} y(\vec{x}_1, \vec{w}) = w_0 + w_1 x_{11} + w_2 x_{12} + \dots + w_D x_{1D} \\ y(\vec{x}_2, \vec{w}) = w_0 + w_1 x_{21} + w_2 x_{22} + \dots + w_D x_{2D} \\ \dots \\ y(\vec{x}_N, \vec{w}) = w_0 + w_1 x_{N1} + w_2 x_{N2} + \dots + w_D x_{ND} \end{cases}$$
$$\begin{cases} y(\vec{x}_1, \vec{w}) = \vec{x}_1^T \vec{w} & \text{where } \vec{x}_1^T = [1, x_{11}, x_{12}, \dots, x_{1D}] \\ y(\vec{x}_2, \vec{w}) = \vec{x}_2^T \vec{w} & \text{where } \vec{x}_2^T = [1, x_{21}, x_{22}, \dots, x_{2D}] \\ \dots \\ y(\vec{x}_N, \vec{w}) = \vec{x}_N^T \vec{w} & \text{where } \vec{x}_N^T = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \end{cases}$$

Matrix Notation



3689302

$$\begin{cases} y(\vec{x}_1, \vec{w}) = \vec{x}_1^T \vec{w} & \text{where } \vec{x}_1^T = [1, x_{11}, x_{12}, \dots, x_{1D}] \\ y(\vec{x}_2, \vec{w}) = \vec{x}_2^T \vec{w} & \text{where } \vec{x}_2^T = [1, x_{21}, x_{22}, \dots, x_{2D}] \\ \dots \\ y(\vec{x}_N, \vec{w}) = \vec{x}_N^T \vec{w} & \text{where } \vec{x}_N^T = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \end{cases}$$

$$\begin{cases} y(\vec{x}_1, \vec{w}) = [1, x_{11}, x_{12}, \dots, x_{1D}] \vec{w} \\ y(\vec{x}_2, \vec{w}) = [1, x_{21}, x_{22}, \dots, x_{2D}] \vec{w} \\ \dots \\ y(\vec{x}_N, \vec{w}) = [1, x_{N1}, x_{N2}, \dots, x_{ND}] \vec{w} \end{cases}$$

$$\mathbb{X} = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N\} \in \mathbb{R}^{N \times (D+1)}$$

Design matrix

$$\mathbb{Y} = \{y(\vec{x}_1, \vec{w}), y(\vec{x}_2, \vec{w}), \dots, y(\vec{x}_N, \vec{w})\} \in \mathbb{R}^N$$

$$\mathbb{Y}(\mathbb{X}, \vec{w}) = \mathbb{X} \vec{w}$$

$$\in \mathbb{R}^{N \times (D+1)}$$

$$\in \mathbb{R}^{D+1}$$

Error Function

- Gaussian Mixture Model
- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation



3689302

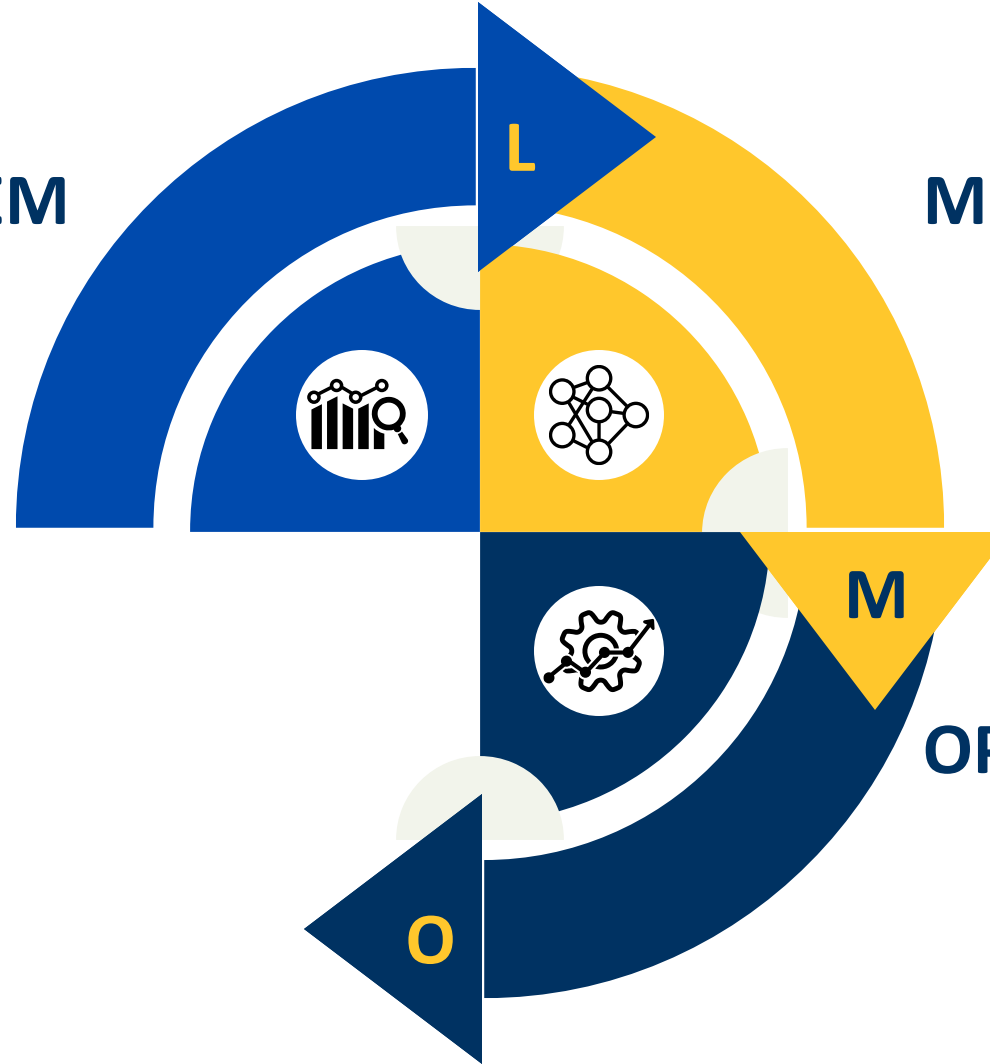
Optimization



3689302

LEARNING PROBLEM

Supervised learning
of scalar target values



MODEL DESIGN

Linear Regression
with basis functions
 $y(\vec{x}, \vec{w}) = \vec{x}^T \vec{w}$

OPTIMIZATION

Derive direct solution

$$E(w) = \frac{1}{2} \sum_{n=1}^N (t_n - y(\mathbf{x}_n, w))^2$$
$$w^* = \underset{w}{\operatorname{argmin}} E(w)$$



3689302

Error Function

- How to calculate the weights $\vec{w}$?
- We do this by minimizing an **error function** that measures the **misfit** between the function $\mathbb{Y}(\mathbb{X}, \vec{w})$ and **targets**, for any given value of $\vec{w}$, and the training data $\mathbb{X}$.
- One simple error function: **Sum of the squares of the differences between targets and the predictions.**

$$E(w) = \frac{1}{2} \sum_{n=1}^N (t_n - y(x_n, w))^2$$

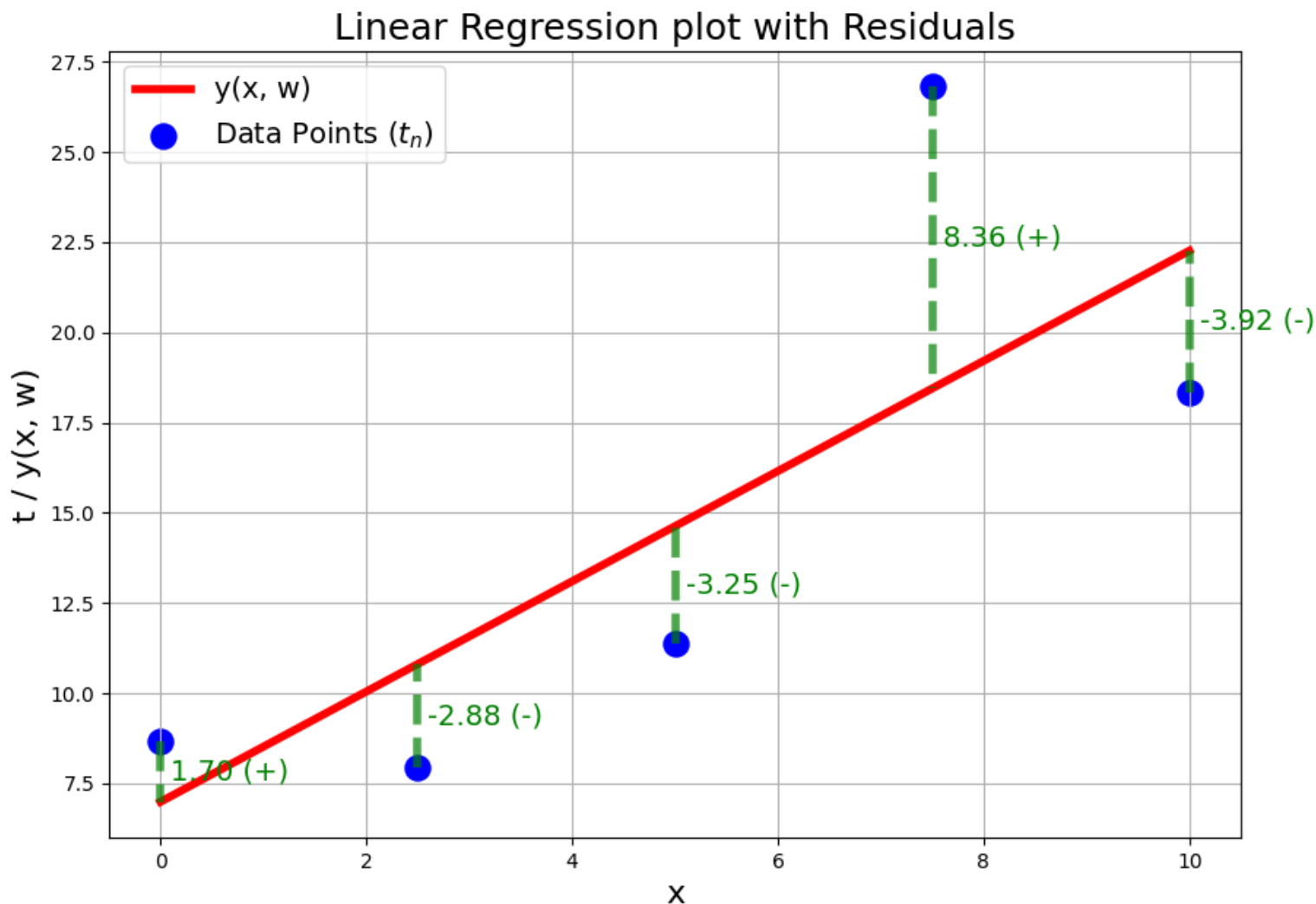
- Non-negative quantity.
- Only 0 if all predictions are equal to targets.

The factor of $\frac{1}{2}$ is included for later convenience.

Error Function Visualization

Residuals are defined as
 $e_n = (t_n - y(x_n, w))$
and they can be + or -.

The error function $E(w)$
combines the squares of
the residuals to measure
the goodness of the fit.



Error Function Minimization

- Gaussian Mixture Model
- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation



3689302



3689302

Error Function Minimization

- $E(\mathbf{w})$ is a quadratic function.

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2$$

Therefore, $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}$ is a linear function of $\mathbf{w}$ and hence solving $\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0$ has one and only one solution. Let's now derive this solution:

$$= \frac{1}{2} \sum_{n=1}^N \left(\mathbf{t}_n - \mathbf{w}_0 - \sum_{j=1}^D \mathbf{w}_j \mathbf{x}_{nj} \right)^2 \quad \xrightarrow{\text{Finding the optimum solution}} \quad \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0$$



3689302

Error Function Minimization

$$\frac{\partial E(w)}{\partial w_0} = 0 \longrightarrow \frac{\partial \left(\frac{1}{2} \sum_{n=1}^N \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd}) \right)^2 \right)}{\partial w_0} = 0$$

Sum rule

$$\frac{\partial \sum f}{\partial w_0} = \sum \frac{\partial f}{\partial w_0} = \frac{1}{2} \sum_{n=1}^N \frac{\partial \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd})^2 \right)}{\partial w_0}$$

Chain rule
 $\frac{\partial f^2}{\partial f} = 2f \frac{\partial f}{\partial w_0}$

$$= \frac{1}{2} \sum_{n=1}^N 2 \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) \left(\frac{\partial (t_n - w_0 - \sum_{d=1}^D w_d x_{nd})}{\partial w_0} \right)$$

Separating
the terms

$$= \sum_{n=1}^N \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) (-1) = - \left(\sum_{n=1}^N t_n - n w_0 - \sum_{n=1}^N \left(\sum_{d=1}^D w_d x_{nd} \right) \right)$$

$$\frac{\partial E(w)}{\partial w_0} = 0 \longrightarrow w_0 = \frac{1}{n} \left(\sum_{n=1}^N t_n - \sum_{n=1}^N \left(\sum_{d=1}^D w_d x_{nd} \right) \right)$$



3689302

Error Function Minimization

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}_k} = 0 \quad \longrightarrow \quad \frac{\partial \left(\frac{1}{2} \sum_{n=1}^N \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd}) \right)^2 \right)}{\partial \mathbf{w}_k} = 0$$

Sum rule

$$\frac{\partial \sum f}{\partial \mathbf{w}_k} = \sum \frac{\partial f}{\partial \mathbf{w}_k} = \frac{1}{2} \sum_{n=1}^N \frac{\partial \left((t_n - w_0 - \sum_{d=1}^D w_d x_{nd})^2 \right)}{\partial \mathbf{w}_k}$$

Chain rule

$$\frac{\partial f^2}{\partial f} = 2f \frac{\partial f}{\partial \mathbf{w}_k} \quad = \frac{1}{2} \sum_{n=1}^N 2 \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right) \left(\frac{\partial (t_n - w_0 - \sum_{d=1}^D w_d x_{nd})}{\partial \mathbf{w}_k} \right)$$

Reordering

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}_k} = - \sum_{n=1}^N x_{nk} \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right)$$



3689302

Error Function Minimization

$$\frac{\partial E(w)}{\partial w_k} = - \sum_{n=1}^N x_{nk} \left(t_n - w_0 - \sum_{d=1}^D w_d x_{nd} \right)$$

$$\sum_{n=1}^N x_{nk} t_n = \sum_{n=1}^N x_{nk} \left(w_0 + \sum_{d=1}^D w_d x_{nd} \right) \rightarrow \underbrace{\sum_{n=1}^N x_{nk} t_n}_{\mathbb{X}^T T} = \underbrace{w_0 \sum_{n=1}^N x_{nk} + \sum_{d=1}^D w_d \sum_{n=1}^N x_{nk} x_{nd}}_{\mathbb{X}^T \mathbb{X} w}$$

Normal equations for the least squares problem

$$\mathbb{X}^T T = \mathbb{X}^T \mathbb{X} w$$

Multiplying both sides by $(\mathbb{X}^T \mathbb{X})^{-1}$

$$(\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T = w$$

Takeaway $\rightarrow w^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T$

Assuming $\mathbb{X}^T \mathbb{X}$ is invertible



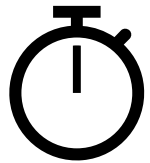
3689302

Solving Normal Equation Runtime

$$\mathbb{X} = \{\vec{x}_1, \vec{x}_2, \dots, \vec{x}_N\} = \underbrace{\begin{bmatrix} 1 & x_{11} & \dots & x_{1D} \\ 1 & x_{21} & \dots & x_{2D} \\ \dots & \dots & \dots & \dots \\ 1 & x_{N1} & \dots & x_{ND} \end{bmatrix}}_{D+1} \begin{matrix} N \\ \end{matrix} \in \mathbb{R}^{N \times (D+1)}$$

$\downarrow$

$$\mathbb{X}^T \mathbb{X} \in \mathbb{R}^{(D+1) \times (D+1)}$$



Time complexity

Forming $\mathbb{X}^T \mathbb{X}$: $\mathcal{O}(N^2 D^2)$

Forming $\mathbb{X}^T T$: $\mathcal{O}(N^2 D)$

Forming $(\mathbb{X}^T \mathbb{X})^{-1}$: $\mathcal{O}(D^3)$

Calculating $(\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T T$: $\mathcal{O}(D^3)$

$\longrightarrow \mathcal{O}(N^2 D^2 + D^3)$

Geometric Interpretation

- Gaussian Mixture Model
- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation



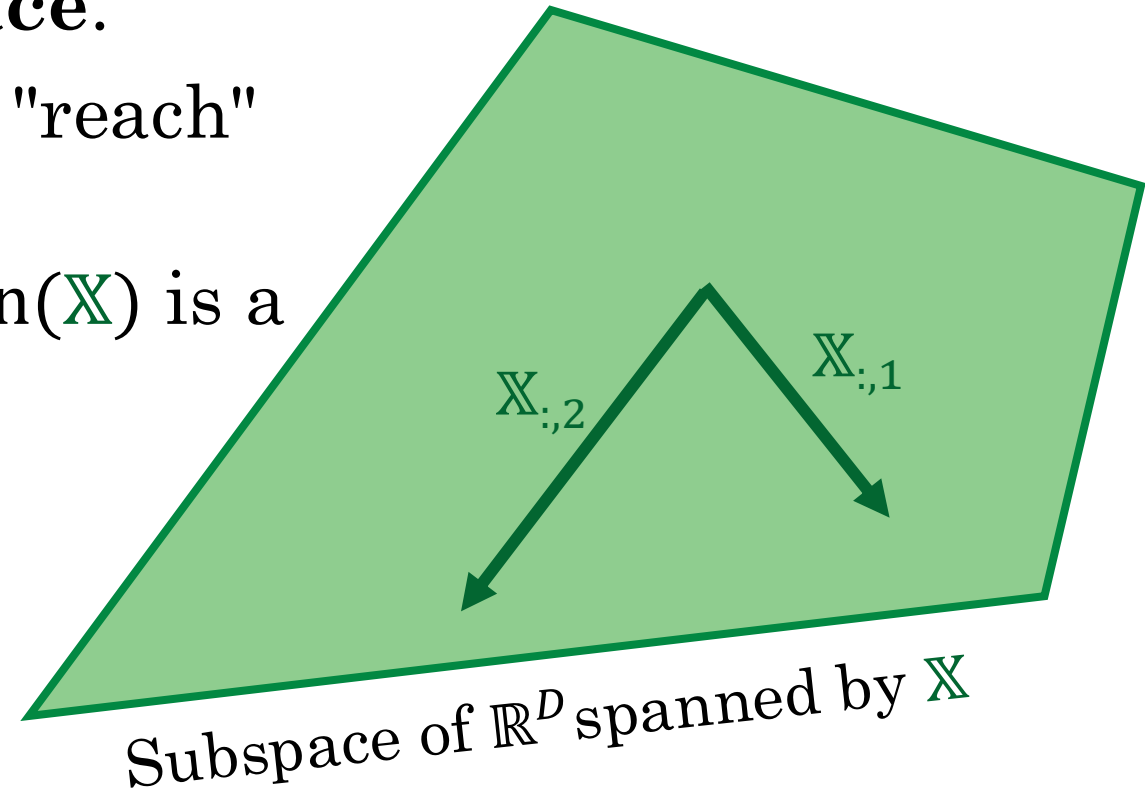
3689302



3689302

[Linear Algebra] Span

- The set of all possible linear combinations of the columns of $\mathbb{X}$ is called the **span** of $\mathbb{X}$ (denoted $\text{span}(\mathbb{X})$), also called the **column space**.
- Intuitively, this is all vectors you can "reach" using the columns of $\mathbb{X}$.
- If each column of $\mathbb{X}$ has length D , $\text{span}(\mathbb{X})$ is a subspace of $\mathbb{R}^D$.



[Linear Algebra] Matrix-Vector Multiplication



3689302

- **Approach 1:** So far, we've thought of our model as horizontally stacked predictions per datapoint:

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} \text{---} & \mathbf{x}_1^T & \text{---} \\ \text{---} & \mathbf{x}_2^T & \text{---} \\ & \vdots & \\ \text{---} & \mathbf{x}_N^T & \text{---} \end{bmatrix} \begin{bmatrix} w \end{bmatrix}$$

Matrix sizes
 $N \times 1 = (N \times (D + 1)) ((D + 1) \times 1)$

- **Approach 2:** However, it is helpful sometimes to think of matrix-vector multiplication as performed by columns. We can also think of $\mathbb{Y}$ as a **linear combination of feature vectors**, scaled by **parameters**.

$$\begin{bmatrix} | \\ \mathbb{Y} \\ | \end{bmatrix} = \begin{bmatrix} | & & | \\ \mathbb{X}_{:,0} & \dots & \mathbb{X}_{:,D} \\ | & & | \end{bmatrix} \begin{bmatrix} w \end{bmatrix} = w_0 \begin{bmatrix} | \\ \mathbb{X}_{:,0} \\ | \end{bmatrix} + w_1 \begin{bmatrix} | \\ \mathbb{X}_{:,1} \\ | \end{bmatrix} + \dots + w_D \begin{bmatrix} | \\ \mathbb{X}_{:,D} \\ | \end{bmatrix}$$

Prediction Is a Linear Combination of Columns

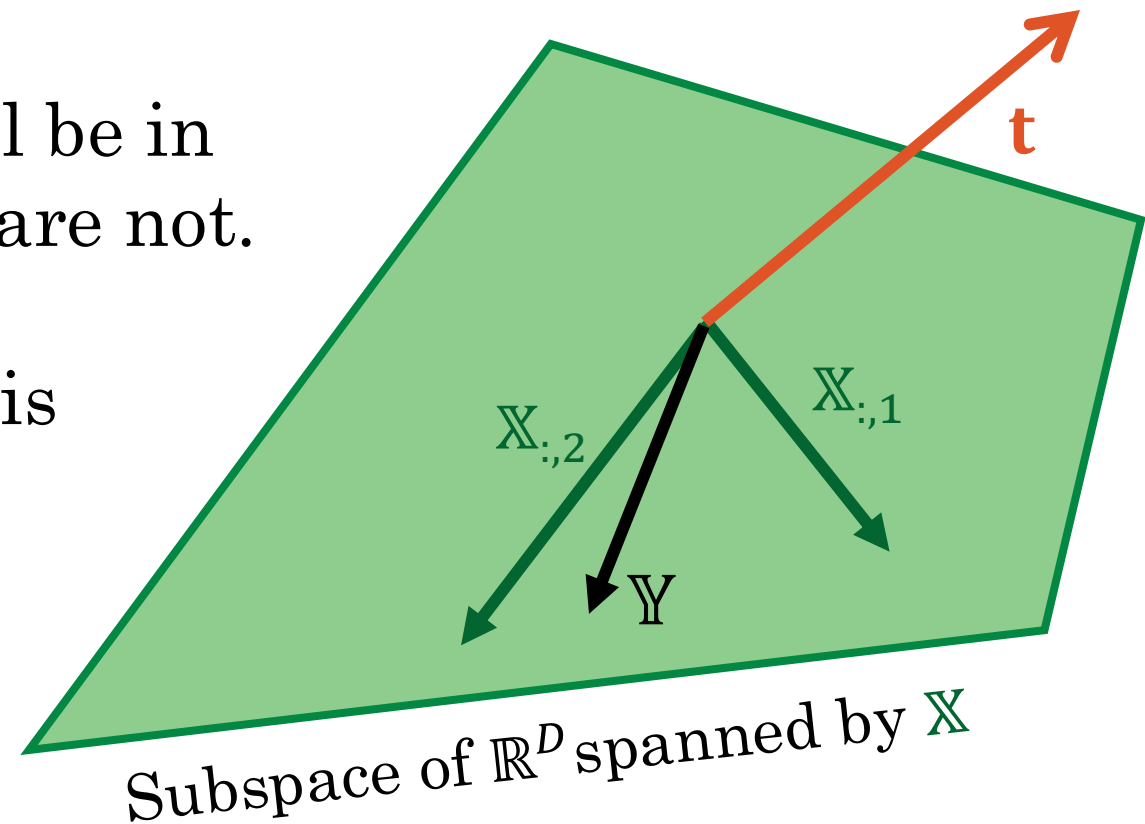


3689302

Our prediction of $\mathbb{Y}(\mathbb{X}, \vec{w}) = \mathbb{X} \vec{w}$ is a linear combination of columns of $\mathbb{X}$.

Interpret: Our linear prediction $\mathbb{Y}$ will be in $\text{span}(\mathbb{X})$, even if ground-truth values $\mathbf{t}$ are not.

Goal: Find vector of $\mathbb{Y}$ in $\text{span}(\mathbb{X})$ that is closest to $\mathbf{t}$.





**What's the geometry word for
'closest point in a subspace'?**

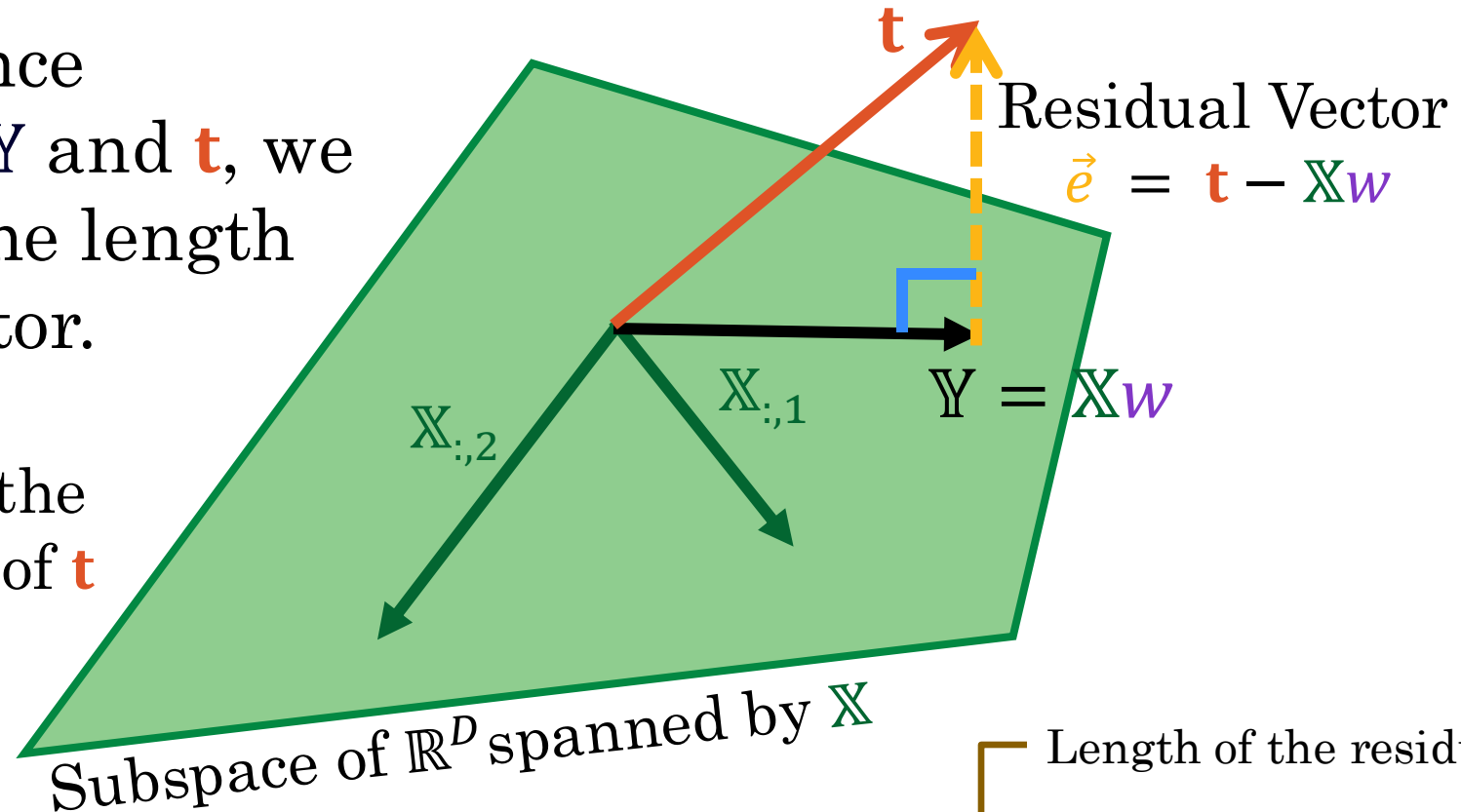


3689302

Finding Optimum Predictions ($\mathbb{Y}$)

To minimize distance between vector of $\mathbb{Y}$ and $\mathbf{t}$, we should minimize the length of the residual vector.

$\vec{e}$ is minimized if it is the orthogonal projection of $\mathbf{t}$ on the $\text{span}(\mathbb{X})$.



Length of the residual vector

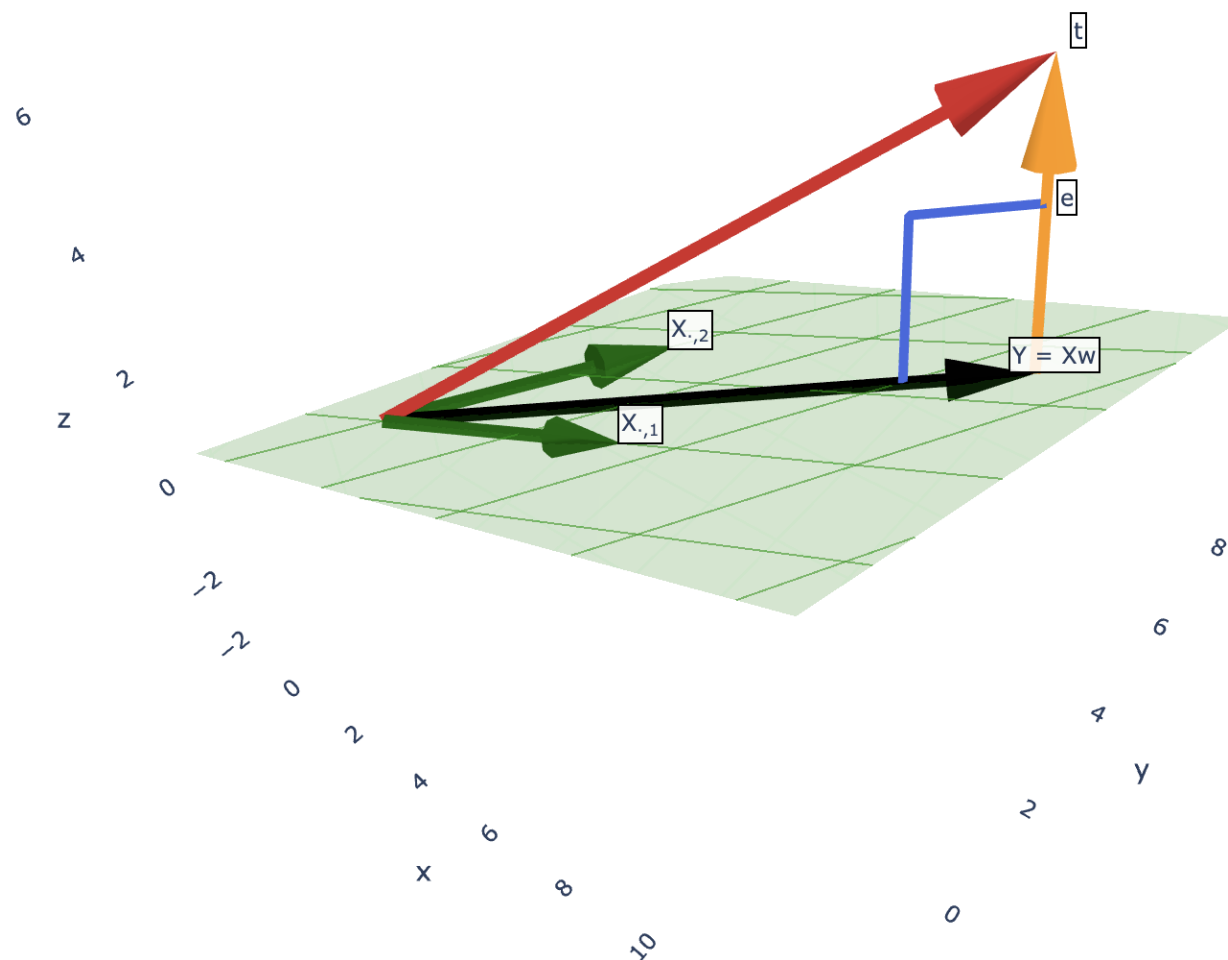
$$\begin{aligned} L_2 - \text{norm}(\vec{e}) &= \|\vec{e}\|_2 \\ &= \sqrt{\sum_{d=0}^D e_d^2} \end{aligned}$$

Geometry of Least Squares in Plotly

[Interactive link](#)



3689302

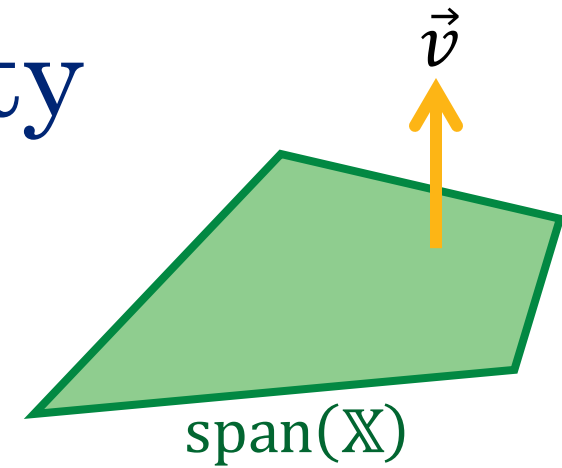


[Linear Algebra] Orthogonality



3689302

- A vector $\vec{v}$ is orthogonal to the $\text{span}(\mathbb{X})$ if and only if it is orthogonal to all columns of $\mathbb{X}$.



Assuming $\vec{v} \in \mathbb{R}^N$ and $\mathbb{X} = \begin{bmatrix} | & & | \\ \mathbb{X}_{:,0} & \dots & \mathbb{X}_{:,D} \\ | & & | \end{bmatrix} \in \mathbb{R}^{N \times D}$

$\vec{v}$ orthogonal to each column means:

$$\mathbb{X}_{:,0}^T \vec{v} = 0$$

$$\mathbb{X}_{:,1}^T \vec{v} = 0$$

$$\vdots$$

$$\mathbb{X}_{:,D}^T \vec{v} = 0$$

$$\Rightarrow \begin{bmatrix} \mathbb{X}_{:,0}^T \vec{v} \\ \mathbb{X}_{:,1}^T \vec{v} \\ \vdots \\ \mathbb{X}_{:,D}^T \vec{v} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$$\Rightarrow \mathbb{X}^T \vec{v} = 0$$

We will use
this shortly



3689302

Going Back to Our Error Function

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N (\mathbf{t}_n - y(\mathbf{x}_n, \mathbf{w}))^2 = \frac{1}{2} (\|\vec{e}\|_2)^2$$

Equivalently, vector $\vec{e}$ is orthogonal to the $\text{span}(\mathbb{X})$, meaning that:

$$\mathbb{X}^T \vec{e} = 0 \quad \text{Adding the definition of residual}$$

$$\mathbb{X}^T (\mathbf{t} - \mathbb{Y}(\mathbb{X}, \mathbf{w}^*)) = 0 \quad \text{Adding formula for } \mathbb{Y}(\mathbb{X}, \mathbf{w}^*) = \mathbb{X}\mathbf{w}^*$$

$$\mathbb{X}^T \mathbf{t} - \mathbb{X}^T \mathbb{X} \mathbf{w}^* = 0 \quad \text{Moving terms}$$

$$\mathbb{X}^T \mathbf{t} = \mathbb{X}^T \mathbb{X} \mathbf{w}^*$$

Normal
Equation

If $\mathbb{X}^T \mathbb{X}$ is
invertible

$$\mathbf{w}^* = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T \mathbf{t}$$

Evaluation

- Gaussian Mixture Model
- Linear Regression
- Basis Functions
- Vectorizing Calculations
- Error Function
- Error Function Minimization
- Geometric Interpretation
- Evaluation

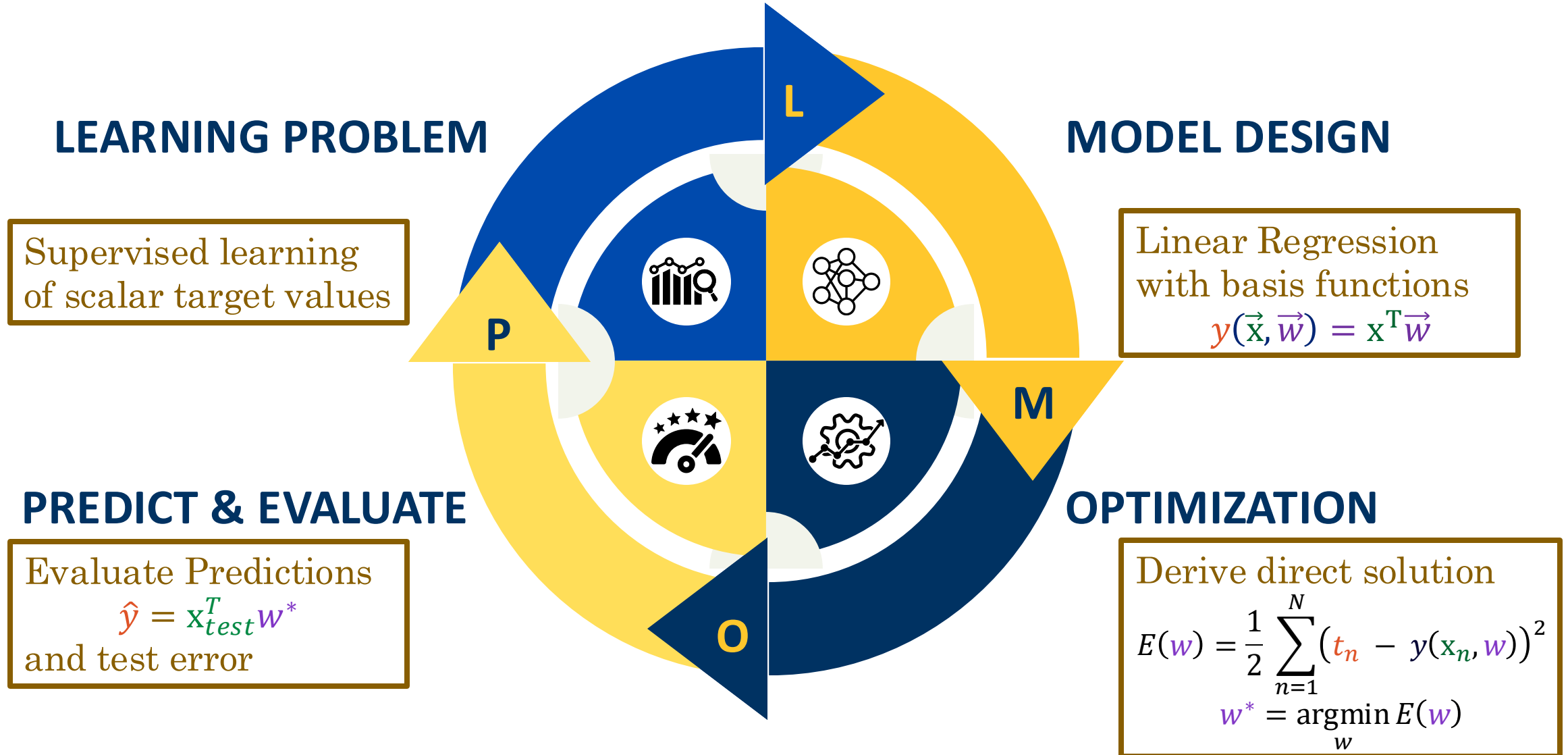


3689302

Predict and Evaluate



3689302





3689302

Evaluation - Visualization

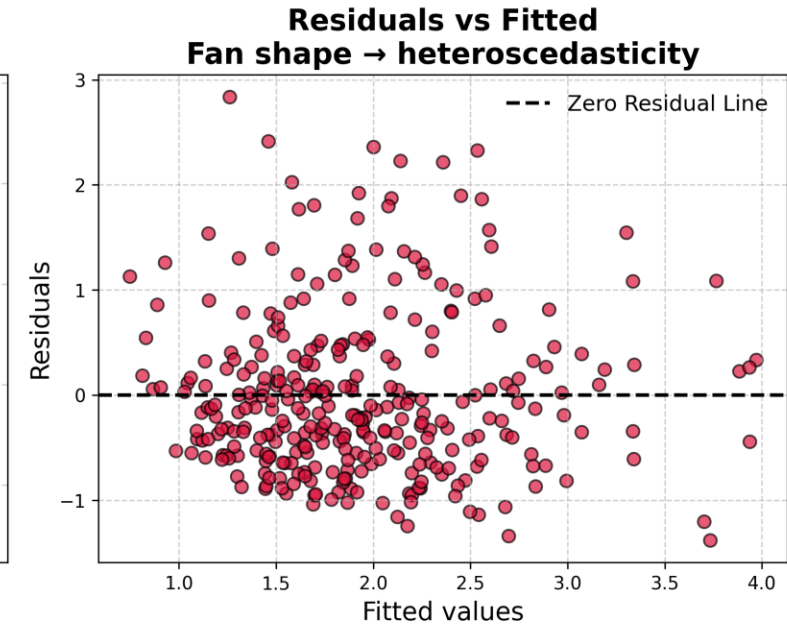
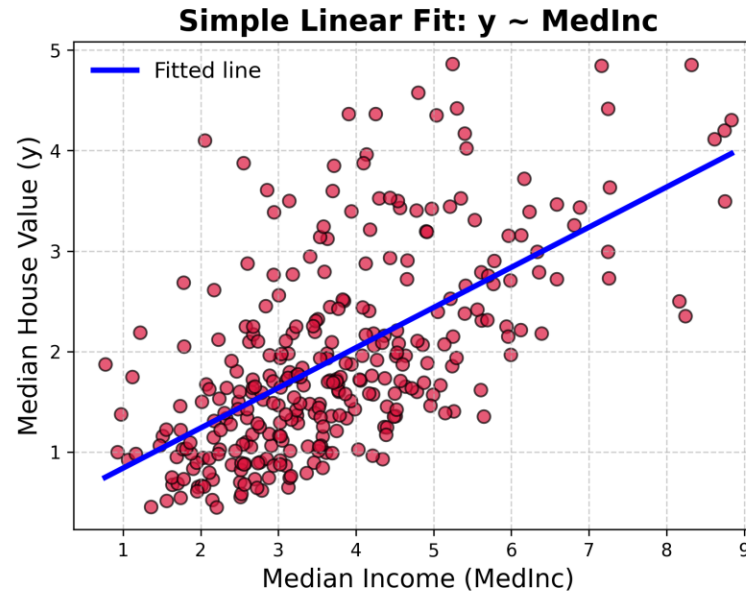
Residual plot shows the trend of the residuals $e_n = (t_n - y(x_n, w))$ with respect to predictions $y(x_n, w)$.

What a good residual plot looks like:

- Points scattered randomly around 0 (no pattern).

- Roughly constant vertical spread (homoscedasticity).

- No obvious trend with fitted values or with predictors.





When you see a fan shape in the residual plot, what comes to mind?

Evaluation - Metrics



3689302

Evaluation - Metrics



3689302

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Evaluation - Metrics



3689302

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error
(RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

Moves the metric
back to the original
unit of the data
compared to MSE

Evaluation - Metrics



3689302

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error
(RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

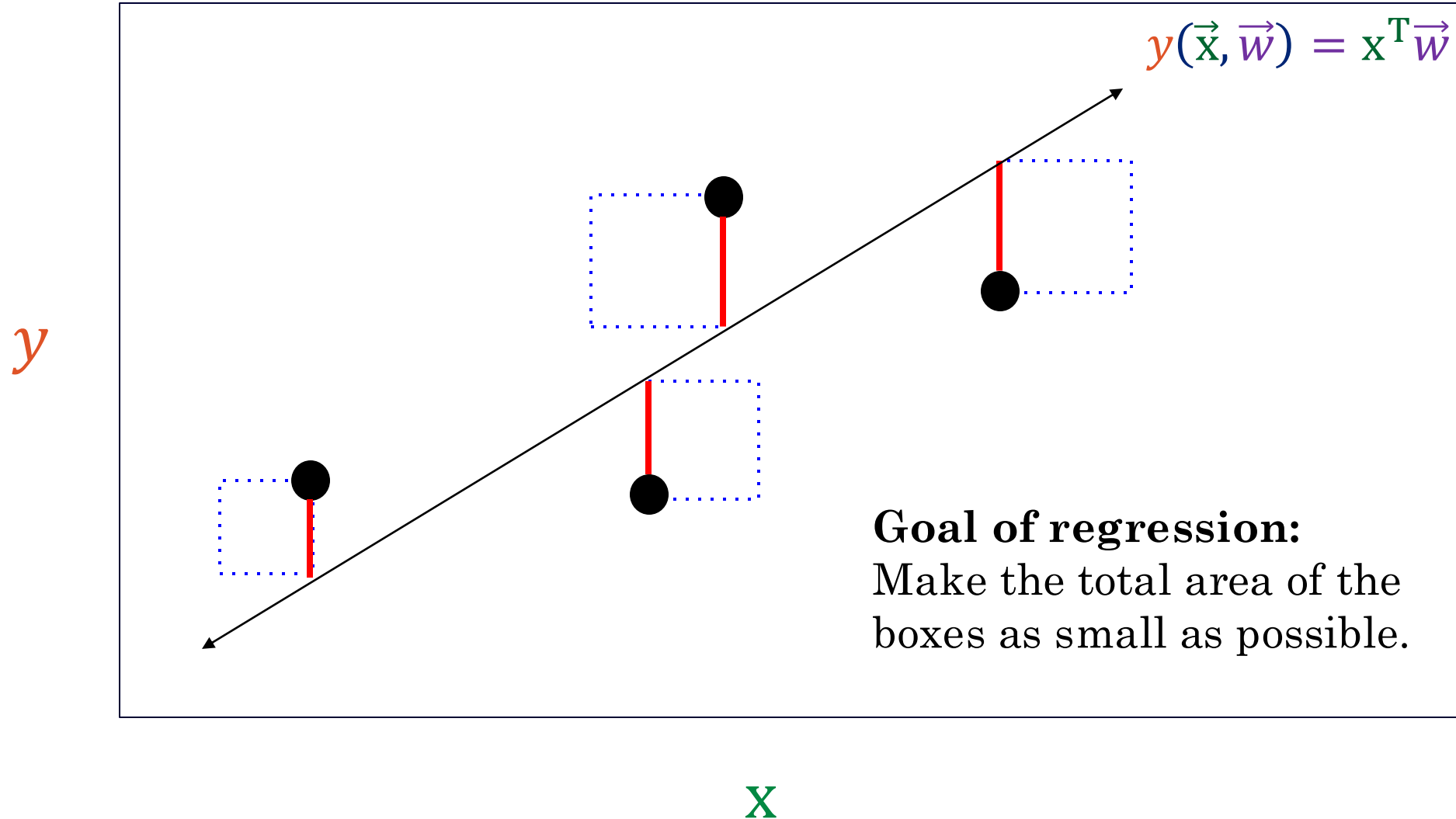
R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Visualizing the Sum of Squared Error of Regression Model



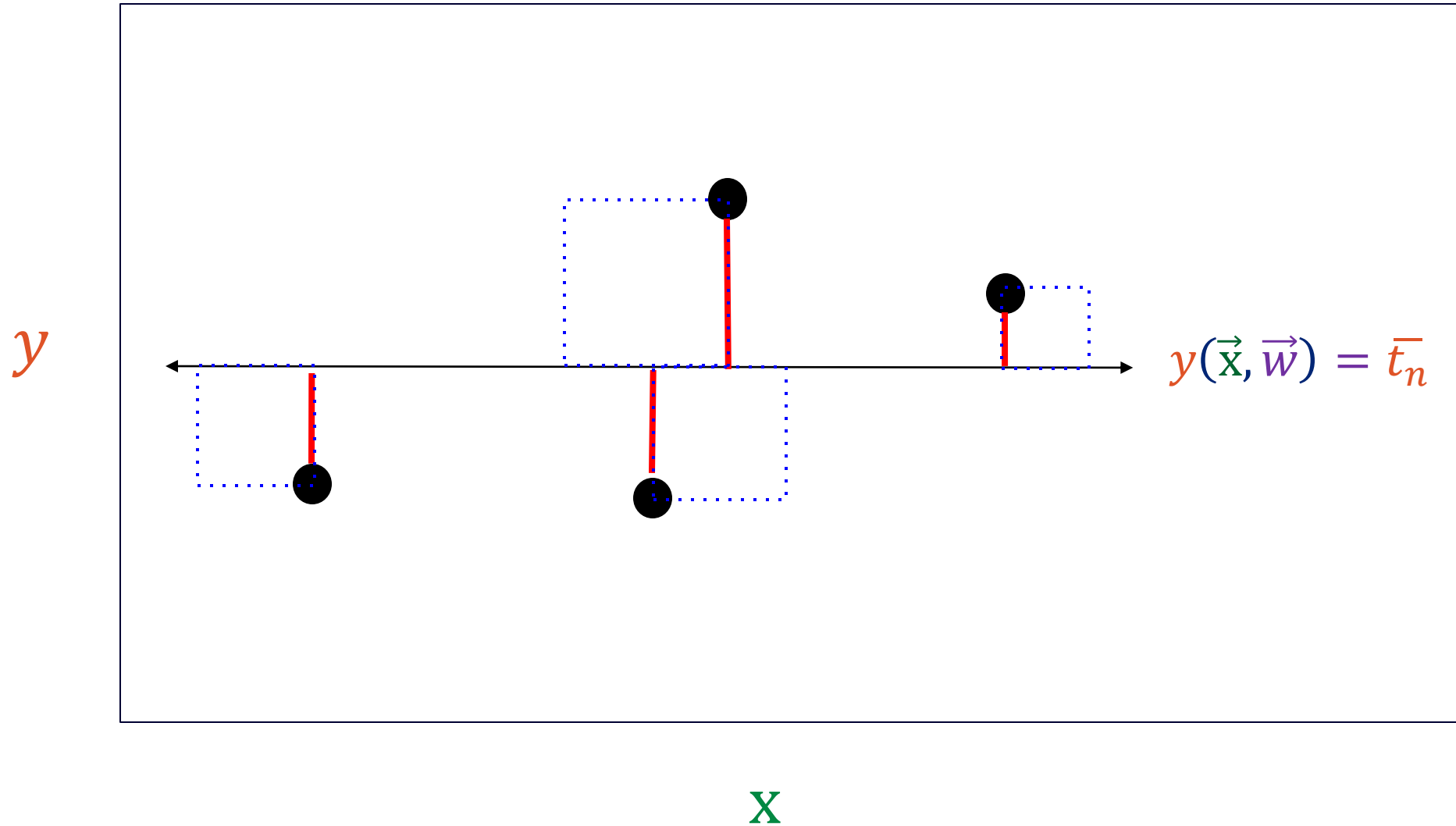
3689302



Visualizing the Sum of Squared Error of Intercept Model



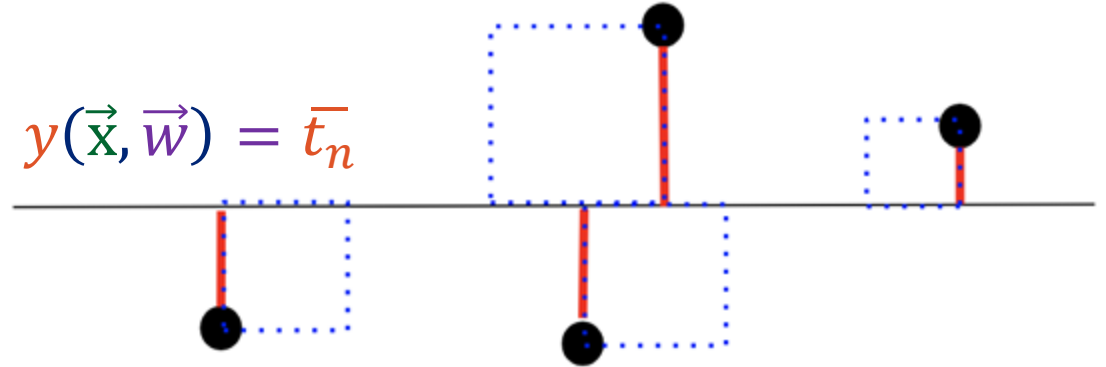
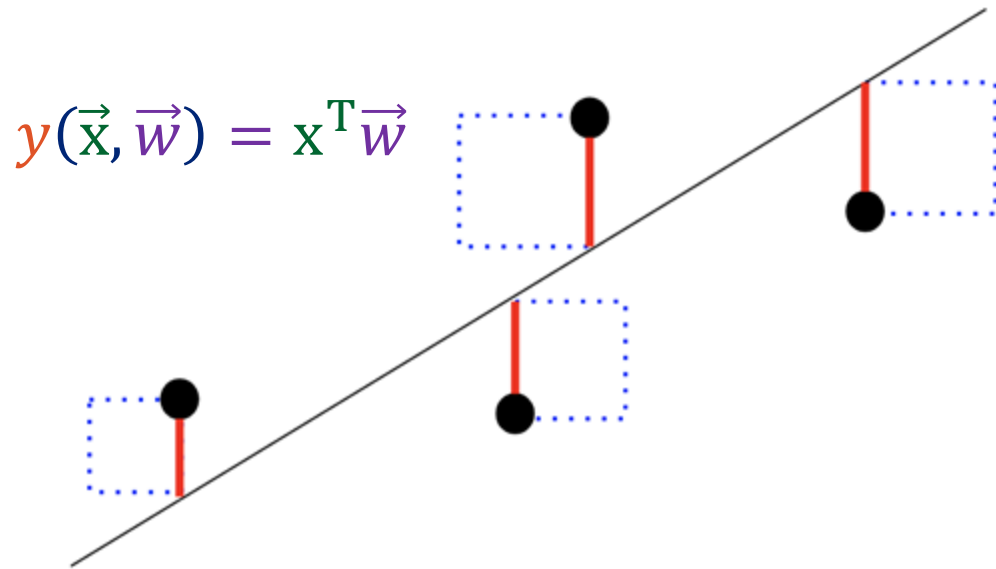
3689302



R²: Quality of the Fit Relative to Intercept Model



3689302



$$R^2 = \frac{\Delta \text{ in area}}{\text{Constant Model Area}} = 1 - \frac{\sum_{n=1}^N (t_n - y(\vec{x}_n, \vec{w}))^2}{\sum_{n=1}^N (t_n - \bar{t}_n)^2}$$

unitless and only compares performance relative to mean baseline.



3689302

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(\mathbf{x}_n, \mathbf{w})| \right)$$

In the same unit as the data; similar to MSE but differs in how the penalization applies.



3689302

Evaluation - Metrics

Mean Squared Error (MSE)

$$\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)$$

Root Mean Squared Error (RMSE)

$$\sqrt{\frac{1}{N} \left(\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2 \right)}$$

R-Squared (R^2) Score

$$1 - \frac{\sum_{n=1}^N (t_n - y(\mathbf{x}_n, \mathbf{w}))^2}{\sum_{n=1}^N (t_n - \bar{t})^2}$$

Mean Absolute Error (MAE)

$$\frac{1}{N} \left(\sum_{n=1}^N |t_n - y(\mathbf{x}_n, \mathbf{w})| \right)$$

Mean Absolute Percentage Error (MAPE)

$$\frac{1}{N} \left(\sum_{n=1}^N \frac{t_n - y(\mathbf{x}_n, \mathbf{w})}{t_n} \right) \times 100$$

Lecture 6

Linear Regression (1)

Credit: Joseph E. Gonzalez and Narges Norouzi

Reference Book Chapters: Chapter 1.[2.1-2.3], Chapter 4.[1.1, 1.4]